Documento de diseño Entrega 1 Proyecto Risk

Saúl Leonardo Cruz Romero, Isabella Hermosa Losada y Alejandro Macías Barrios

Pontificia Universidad Javeriana, Bogotá D.C., Colombia
sa.cruz@javeriana.edu.co | ishermosa@javeriana.edu.co | amaciasb@javeriana.edu.co
Estructuras de Datos - 2026

Resumen. Este documento presenta el diseño del Componente 1 del proyecto Risk, encargado de inicializar una partida y controlar su desarrollo por turnos mediante estructuras lineales. Se describen las entradas, condiciones, proceso y resultados de los comandos inicializar, obtener_unidades, atacar, fortificar, estado_juego, ayuda y salir. También se documentan los Tipos Abstractos de Datos utilizados para representar jugadores, territorios, continentes, cartas, dados, turnos, ataques y el estado general de la partida. El diseño se relaciona con la implementación actual en C++ y deja espacios definidos para los diagramas de flujo y las evidencias de ejecución. Finalmente, se propone un plan de pruebas para el comando atacar con casos normales, límite y de error. Los resultados que todavía no han sido comprobados se conservan como pendientes, con el fin de no afirmar comportamientos sin una ejecución real.

Palabras clave: Risk, estructuras lineales, tipos abstractos de datos, C++, pruebas de software.

1 Descripción del problema

Risk es un juego de mesa por turnos en el que pueden participar entre tres y seis jugadores. Cada jugador se identifica con un nombre y un color diferente, y recibe una cantidad inicial de unidades de acuerdo con la cantidad de participantes: 35 unidades para tres jugadores, 30 para cuatro, 25 para cinco y 20 para seis. Estas unidades deben quedar distribuidas entre los territorios que pertenecen a cada jugador antes de comenzar los turnos [1].

El tablero está formado por 42 territorios agrupados en seis continentes: América del Norte, América del Sur, Europa, África, Asia y Oceanía. Cada territorio conserva su propietario, sus unidades y los códigos de sus vecinos. Estas conexiones determinan desde cuáles territorios puede realizarse un ataque o una fortificación [1].

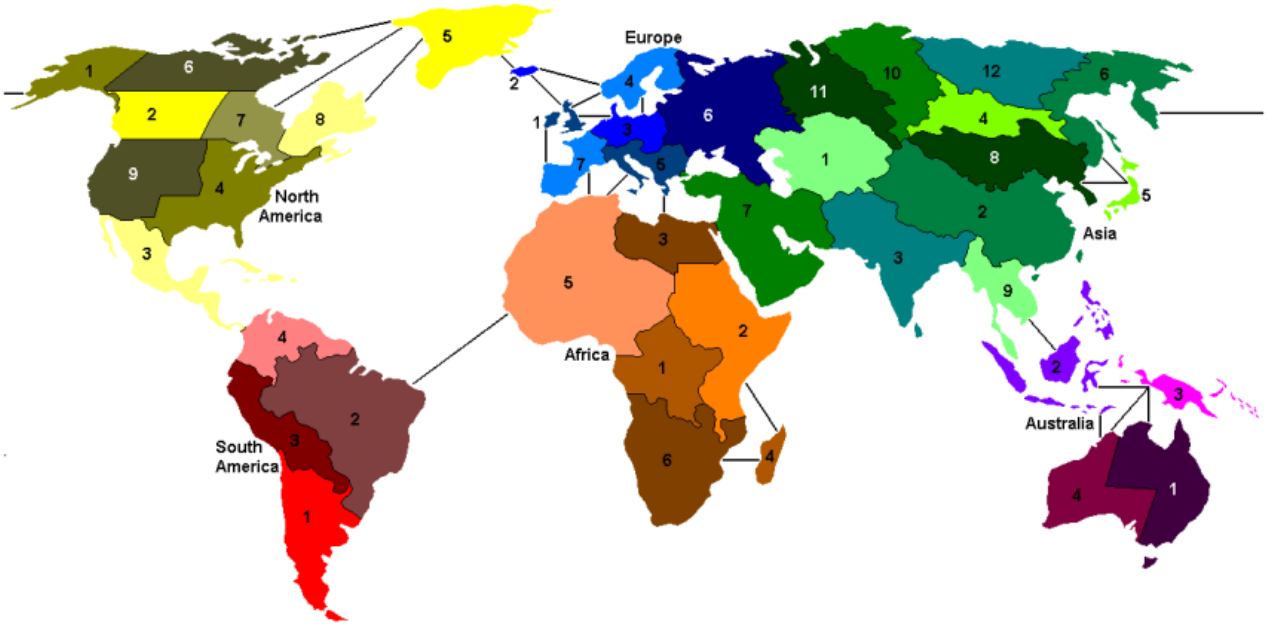


Figura 1. Tablero de juego de Risk. Tomado del enunciado oficial del proyecto [1], que acredita la imagen a Demonaire bajo licencia CC BY-SA 3.0.

Al iniciar el programa, los jugadores y la distribución inicial del tablero se cargan desde un archivo de texto. El programa debe revisar que la cantidad de jugadores sea válida, que los nombres y colores no se repitan, que aparezcan los 42 territorios y que las unidades correspondan con la cantidad de participantes.

Cada turno tiene tres etapas. Primero se obtienen y ubican nuevas unidades; después el jugador puede atacar territorios enemigos vecinos; finalmente puede trasladar unidades entre dos territorios propios vecinos. Los ataques se resuelven comparando dados ordenados de mayor a menor y los empates favorecen al defensor [1].

Las cartas de territorio y los comodines permiten obtener unidades adicionales mediante grupos válidos de tres cartas. Cuando un jugador conquista al menos un territorio durante su etapa de ataque puede recibir una carta. La implementación obligatoria determina la victoria cuando un jugador controla los 42 territorios. La variante de misiones secretas se conserva como bono opcional y todavía no forma parte del código actual.

2 Componente 1 Organización de la información

El Componente 1 organiza la información necesaria para iniciar y desarrollar una partida. Los comandos inicializar, obtener_unidades, atacar, fortificar y estado_juego cargan o modifican el estado del juego, mientras que ayuda y salir permiten utilizar y finalizar la interfaz. La información se distribuye entre TAD que representan jugadores, territorios, continentes, cartas, dados, turnos y ataques.

El componente también controla el orden de las etapas. Un jugador debe ubicar sus unidades antes de atacar y debe terminar el ataque antes de fortificar. Al terminar la fortificación se avanza al siguiente jugador que todavía conserve territorios.

2.1 Descripción de los comandos

INICIALIZAR

Forma de uso: inicializar nombre_archivo

Carga desde un archivo de texto los jugadores y la distribución inicial de los 42 territorios.

Entrada. Recibe un único nombre o ruta de archivo, por ejemplo, inicializar partida.txt.

Condiciones. No debe existir una partida inicializada. El archivo debe abrirse, contener de tres a seis jugadores, nombres de máximo ocho caracteres, colores permitidos y no repetidos, y exactamente los 42 territorios con códigos válidos, propietarios registrados y unidades positivas. El total por jugador debe ser 35, 30, 25 o 20 según participen tres, cuatro, cinco o seis jugadores.

Proceso. Los datos se leen y validan en variables temporales. Cuando todo el archivo es correcto, se guardan los jugadores y el tablero, el primer jugador del archivo queda en la posición inicial del turno y se prepara el mazo.

Resultados. Puede retornar COMANDO_CORRECTO, ARCHIVO_VACIO, ARCHIVO_ERRONEO, ARCHIVO_SIN_FORMATO o JUEGO_EN_CURSO. Si ocurre un error, no se conserva una partida incompleta.

OBTENER_UNIDADES

Forma de uso: obtener_unidades nombre_jugador

Calcula las unidades del jugador y permite repartirlas entre territorios propios.

Entrada. Recibe el nombre del jugador. Durante la ejecución se solicitan los canjes de cartas y pares formados por código de territorio y cantidad.

Condiciones. La partida debe estar activa, el jugador debe existir, tener el turno y encontrarse en la etapa de obtención. Las cantidades deben ser enteras positivas, no superar las unidades pendientes y asignarse a territorios propios.

Cálculo. Se suma la división entera de los territorios controlados entre tres, la bonificación de los continentes completos y la bonificación de cartas. Los canjes válidos son tres cartas del mismo tipo, una de cada tipo o dos cartas de ejército con un comodín. Los valores de canje son 4, 6, 8, 10, 12 y 15; después aumentan de cinco en cinco [1].

Proceso. El jugador puede realizar canjes mientras conserve al menos tres cartas. Si una carta canjeada representa un territorio propio, se agregan dos unidades directamente en ese territorio. Después se reparten todas las unidades pendientes. Los cambios temporales se guardan únicamente al finalizar y la etapa pasa a ataque.

Resultados. Puede retornar COMANDO_CORRECTO, JUEGO_NO_INICIALIZADO, JUEGO_TERMINADO, JUGADOR_NO_VALIDO, JUGADOR_FUERA_DE_TURNO, JUGADOR_YA_OBTUVO_UNIDADES o ENTRADA_CERRADA.

ATACAR

Forma de uso: atacar nombre_jugador

Permite realizar una o más rondas de ataque desde un territorio propio hacia un territorio enemigo vecino.

Entrada. Recibe el nombre del jugador. Después solicita si desea atacar, los códigos de origen y destino, la decisión de continuar y, si existe conquista, la cantidad que se trasladará.

Condiciones. La partida debe estar activa, el jugador debe existir, tener el turno y haber ubicado sus unidades. El origen debe ser propio, el destino enemigo, ambos deben existir y ser vecinos, y el origen debe tener más de una unidad.

Dados. El atacante usa un dado con dos unidades, dos dados con tres unidades y tres dados con cuatro o más. El defensor usa un dado con una unidad y dos dados con dos o más. Los resultados se ordenan de mayor a menor; el defensor pierde cuando el dado atacante es mayor y el atacante pierde cuando su valor es menor o igual [1].

Proceso. Las comparaciones actualizan las unidades. Si el defensor conserva unidades, el atacante decide si continúa. Si llega a cero, el territorio cambia de propietario y se trasladan unidades desde el origen. La implementación entrega como máximo una carta por etapa de ataque y comprueba si el jugador controla los 42 territorios.

Resultados. Puede retornar COMANDO_CORRECTO, estados de partida, JUGADOR_NO_VALIDO, JUGADOR_FUERA_DE_TURNO, JUGADOR_NO_HA_UBICADO_UNIDADES, JUGADOR_YA_TERMINO_ATAQUE o ENTRADA_CERRADA. Una configuración territorial inválida no lanza los dados y permite intentar otro ataque.

FORTIFICAR

Forma de uso: fortificar nombre_jugador

Realiza la última etapa del turno mediante un único traslado entre dos territorios propios vecinos, o permite finalizar escribiendo n.

Entrada. Recibe el nombre del jugador. Después solicita origen, destino y cantidad, o la letra n.

Condiciones. La partida debe estar activa, debe ser el turno del jugador y la etapa debe ser fortificación. Los territorios deben existir, ser diferentes, vecinos y propios. La cantidad debe ser positiva y dejar al menos una unidad en el origen.

Proceso. El traslado se intenta sobre una copia del tablero. Si es válido, se guarda el tablero temporal. Si se escribe n no se modifica el tablero. En ambos casos se avanza al siguiente jugador que conserve territorios y la etapa vuelve a obtención de unidades.

Resultados. Puede retornar COMANDO_CORRECTO, estados de partida, JUGADOR_NO_VALIDO, JUGADOR_FUERA_DE_TURNO, JUGADOR_NO_HA_ATACADO o ENTRADA_CERRADA. Los errores de datos vuelven a solicitar la entrada.

ESTADO_JUEGO

Forma de uso: estado_juego

Consulta la partida sin modificarla. No recibe parámetros.

Condiciones. La partida debe estar inicializada y no haber terminado.

Salida. Muestra la cantidad de jugadores, sus nombres y colores, el jugador actual y, para cada territorio, su continente, nombre, código, propietario y unidades. La salida actual no incluye cartas ni la etapa del turno.

Resultados. Muestra el estado cuando validarEstadoJuego retorna COMANDO_CORRECTO; en caso contrario informa JUEGO_NO_INICIALIZADO o JUEGO_TERMINADO.

AYUDA

Formas de uso: ayuda | ayuda nombre_comando

Muestra la lista de comandos disponibles o la explicación de un comando específico. No requiere una partida inicializada.

Condiciones. Puede recibir cero o un parámetro. La ayuda específica busca el nombre dentro de TABLA_COMANDOS.

Salida. La ayuda general agrupa los comandos actuales en Configuración del juego y General. La ayuda específica muestra descripción, uso, ejemplo y cantidad de parámetros. Si el nombre no existe, se informa que el comando es inválido.

SALIR

Forma de uso: salir

Finaliza el ciclo principal y devuelve el control a la terminal. No recibe parámetros, no requiere una partida inicializada y no genera salida por pantalla [1].

2.2 Diseño

El diseño se presenta mediante las funciones auxiliares de la interfaz, el procedimiento principal, los esquemáticos de los comandos y la especificación de los TAD. La función main no se repite en la tabla de funciones auxiliares porque se representa mediante un diagrama de flujo, siguiendo el criterio utilizado en el Taller 2 [4].

Tabla de funciones auxiliares

Las funciones de Comandos.cpp separan la entrada, validan los comandos y muestran la ayuda de la aplicación.

Descripción de los Tipos Abstractos de Datos

Los Tipos Abstractos de Datos se describen mediante el conjunto mínimo de datos que forma su estado y las operaciones que componen su interfaz. Para cada variable se indican sus condiciones cuando corresponde. Cada operación presenta sus argumentos, el valor que retorna, su función, las precondiciones y las poscondiciones, siguiendo la plantilla trabajada en clase [2].

TAD Territorio

Representa un territorio del tablero, junto con su identificación, propietario, cantidad de unidades y conexiones con otros territorios.

Estado

- Código del territorio (codigo), Cadena de caracteres. Identifica el territorio.
- Condición: Puede estar vacío únicamente mientras el territorio no haya sido configurado. En un territorio del tablero debe corresponder con uno de los 42 códigos definidos.
- Nombre del territorio (nombre), Cadena de caracteres. Guarda el nombre con el que se presenta el territorio.
- Condición: Puede estar vacío únicamente mientras el territorio no haya sido configurado.
- Código del continente (codigoContinente), Cadena de caracteres. Identifica el continente al que pertenece el territorio.
- Condición: En un territorio configurado debe corresponder con uno de los seis continentes del tablero.
- Color del propietario (colorPropietario), Cadena de caracteres. Identifica al jugador que controla el territorio.
- Condición: Puede estar vacío antes de cargar la partida. Después de configurar el tablero debe corresponder con el color de uno de los jugadores.
- Cantidad de unidades (unidades), Numérico. Indica cuántas unidades se encuentran en el territorio.
- Condición: unidades ≥ 0.

- Códigos de los vecinos (codigosVecinos), Secuencia de Cadenas de caracteres. Guarda los códigos de los territorios conectados directamente.
- Condición: Cada código debe identificar un territorio vecino definido en el tablero.

Interfaz

Territorio(): Territorio. Crea un territorio vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un territorio con sus datos de identificación y propietario vacíos, cero unidades y sin vecinos registrados.

Territorio(nuevoCodigo, nuevoNombre, nuevoContinente): Territorio. Crea un territorio con los datos de identificación recibidos.

- Precondiciones: nuevoCodigo, nuevoNombre y nuevoContinente deben representar un territorio definido para el tablero.
- Poscondiciones: Se obtiene un territorio con el código, nombre y continente recibidos. El propietario queda vacío, la cantidad de unidades queda en cero y no se registran vecinos.

obtenerCodigo(): Cadena de caracteres. Consulta el código del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de codigo sin modificar el territorio.

obtenerNombre(): Cadena de caracteres. Consulta el nombre del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de nombre sin modificar el territorio.

obtenerCodigoContinente(): Cadena de caracteres. Consulta el código del continente al que pertenece el territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de codigoContinente sin modificar el territorio.

obtenerColorPropietario(): Cadena de caracteres. Consulta el color del propietario.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de colorPropietario sin modificar el territorio.

obtenerUnidades(): Numérico. Consulta la cantidad de unidades del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de unidades sin modificar el territorio.

asignarPropietario(nuevoColor): Vacío. Asigna directamente el propietario del territorio.

- Precondiciones: nuevoColor debe corresponder con el color que se desea cargar.
- Poscondiciones: colorPropietario queda igual a nuevoColor. Los demás datos conservan su valor.

asignarUnidades(nuevaCantidad): Vacío. Asigna directamente la cantidad de unidades del territorio.

- Precondiciones: nuevaCantidad ≥ 0.
- Poscondiciones: unidades queda igual a nuevaCantidad. Los demás datos conservan su valor.

agregarUnidades(cantidad): Booleano. Intenta agregar unidades al territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si cantidad > 0, aumenta unidades en la cantidad recibida y retorna verdadero. Si cantidad ≤ 0, retorna falso y conserva el estado anterior.

retirarUnidades(cantidad, minimoRestante): Booleano. Intenta retirar unidades sin superar el mínimo que debe permanecer.

- Precondiciones: Ninguna.
- Poscondiciones: Si cantidad > 0 y unidades - cantidad ≥ minimoRestante, descuenta la cantidad y retorna verdadero. En caso contrario, retorna falso y conserva la cantidad de unidades.

agregarVecino(codigoVecino): Vacío. Registra un territorio como vecino.

- Precondiciones: codigoVecino debe identificar un territorio conectado directamente.
- Poscondiciones: El código recibido queda agregado al final de codigosVecinos.

esVecino(codigoTerritorio): Booleano. Comprueba si un territorio se encuentra registrado como vecino.

- Precondiciones: Ninguna.

- Poscondiciones: Retorna verdadero si codigoTerritorio se encuentra en codigosVecinos. En caso contrario, retorna falso. El territorio no se modifica.

obtenerVecinos(): Secuencia de Cadenas de caracteres. Consulta todos los códigos de los territorios vecinos.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la secuencia codigosVecinos sin modificarla.

cambiarPropietario(color, cantidad): Booleano. Registra la conquista del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio tiene cero unidades y cantidad > 0, cambia colorPropietario, agrega la cantidad recibida y retorna verdadero. En caso contrario, retorna falso y conserva el estado anterior.

TAD Continente

Representa una región del tablero que reúne varios territorios y entrega una bonificación cuando un jugador la controla completamente.

Estado

- Código del continente (codigo), Cadena de caracteres. Identifica el continente.
- Condición: Puede estar vacío únicamente mientras el continente no haya sido configurado. En el tablero debe corresponder con uno de los seis códigos definidos.
- Nombre del continente (nombre), Cadena de caracteres. Guarda el nombre visible del continente.
- Condición: Puede estar vacío únicamente mientras el continente no haya sido configurado.
- Bonificación (bonificacion), Numérico. Indica la cantidad de unidades adicionales que entrega el continente.
- Condición: En un continente configurado debe corresponder con la bonificación definida en el enunciado. En un continente vacío su valor es cero.
- Territorios (territorios), Secuencia de Territorio. Contiene los territorios que forman el continente.
- Condición: Cada territorio debe tener un código de continente igual al código del continente que lo contiene.

Interfaz

Continente(): Continente. Crea un continente vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un continente con código y nombre vacíos, bonificación igual a cero y sin territorios.

Continente(nuevoCodigo, nuevoNombre, nuevaBonificacion): Continente. Crea un continente identificado.

- Precondiciones: Los datos deben corresponder con uno de los continentes definidos en el tablero.
- Poscondiciones: Se obtiene un continente con el código, nombre y bonificación recibidos. La secuencia de territorios queda vacía.

obtenerCodigo(): Cadena de caracteres. Consulta el código del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna codigo sin modificar el continente.

obtenerNombre(): Cadena de caracteres. Consulta el nombre del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna nombre sin modificar el continente.

obtenerBonificacion(): Numérico. Consulta la bonificación del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna bonificacion sin modificar el continente.

agregarTerritorio(territorio): Vacío. Agrega un territorio al continente.

- Precondiciones: El código de continente del territorio debe corresponder con el continente que lo recibe.
- Poscondiciones: El territorio queda agregado al final de territorios.

obtenerTerritorios(): Secuencia de Territorio. Consulta los territorios del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la secuencia territorios sin modificarla.

estaControladoPor(colorJugador): Booleano. Comprueba si todos los territorios pertenecen al jugador indicado.

- Precondiciones: Ninguna.

- Poscondiciones: Retorna verdadero si la secuencia no está vacía y todos sus territorios tienen como propietario a colorJugador. En caso contrario, retorna falso.

buscarTerritorio(codigo): Territorio o inexistente. Busca un territorio dentro del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Si encuentra un territorio con el código recibido, retorna ese territorio. Si no lo encuentra, informa que no existe.

TAD Tablero

Representa el mapa completo de Risk mediante los continentes y territorios que participan en la partida.

Estado

- Continentes (continentes), Secuencia de Continente. Contiene las regiones que forman el tablero.
- Condición: Antes de inicializar el tablero puede estar vacía. Después de ejecutar inicializarEstructura, debe contener seis continentes y un total de 42 territorios.

Interfaz

Tablero(): Tablero. Crea un tablero vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un tablero sin continentes.

inicializarEstructura(): Vacío. Construye los continentes, territorios y conexiones del mapa.

- Precondiciones: Ninguna.
- Poscondiciones: Reemplaza el estado anterior por los seis continentes y los 42 territorios definidos para el juego.

buscarTerritorio(codigo): Territorio o inexistente. Busca un territorio en todo el tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Si encuentra un territorio con el código recibido, retorna ese territorio. Si no lo encuentra, informa que no existe.

existeTerritorio(codigo): Booleano. Comprueba si un código identifica un territorio del tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si encuentra el territorio y falso en caso contrario. El tablero conserva su estado.

configurarTerritorio(codigo, colorPropietario, unidades): Booleano. Carga el propietario y las unidades iniciales de un territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio existe, asigna el color y las unidades recibidas y retorna verdadero. Si no existe, retorna falso y conserva el tablero.

sonVecinos(codigoOrigen, codigoDestino): Booleano. Comprueba si dos territorios tienen una conexión directa.

- Precondiciones: Ninguna.
- Poscondiciones: Si ambos territorios existen, retorna el resultado de buscar codigoDestino entre los vecinos de codigoOrigen. Si alguno no existe, retorna falso. El tablero no se modifica.

contarTerritorios(colorJugador): Numérico. Cuenta los territorios pertenecientes al jugador indicado.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la cantidad de territorios cuyo propietario corresponde con colorJugador. El tablero no se modifica.

sumarUnidades(colorJugador): Numérico. Suma las unidades de todos los territorios pertenecientes a un jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la suma de unidades de los territorios cuyo propietario corresponde con colorJugador. El tablero no se modifica.

calcularBonificacionContinentes(colorJugador): Numérico. Calcula la bonificación por continentes controlados.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la suma de las bonificaciones de los continentes controlados completamente por colorJugador. El tablero no se modifica.

agregarUnidades(codigoTerritorio, colorJugador, cantidad): Booleano. Intenta agregar unidades a un territorio propio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio existe, pertenece a colorJugador y cantidad > 0, agrega las unidades y retorna verdadero. En caso contrario, retorna falso y conserva el estado anterior.

trasladarUnidades(codigoOrigen, codigoDestino, colorJugador, cantidad, minimoRestante): Booleano. Intenta trasladar unidades entre dos territorios propios vecinos.

- Precondiciones: Ninguna.
- Poscondiciones: Si ambos territorios existen, son vecinos, pertenecen a colorJugador y el origen puede conservar el mínimo indicado, descuenta la cantidad del origen, la agrega al destino y retorna verdadero. En caso contrario, retorna falso y conserva las unidades de ambos territorios.

cambiarPropietario(codigoTerritorio, nuevoColor, cantidad): Booleano. Intenta registrar la conquista de un territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio existe, pertenece a otro jugador, tiene cero unidades y la cantidad recibida es positiva, cambia su propietario, asigna las nuevas unidades y retorna verdadero. En caso contrario, retorna falso.

todosLosTerritoriosConfigurados(): Booleano. Comprueba si todos los territorios tienen propietario.

- Precondiciones: El tablero debe contener los continentes y territorios del juego.
- Poscondiciones: Retorna verdadero si ningún territorio tiene el propietario vacío. Si encuentra al menos uno sin propietario, retorna falso. El tablero no se modifica.

mostrarTablero(): Vacío. Presenta los continentes y territorios del tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Muestra el nombre, código, propietario y unidades de cada territorio. El tablero conserva su estado.

TAD Carta

Representa una carta de territorio o un comodín utilizado durante los intercambios.

Estado

- Tipo de carta (tipo), TipoCarta. Identifica la figura de la carta.
- Condición: Debe corresponder con infantería, caballería, artillería o comodín.
- Código del territorio (codigoTerritorio), Cadena de caracteres. Identifica el territorio asociado con la carta.
- Condición: Debe corresponder con un territorio del tablero cuando la carta no es un comodín. Para un comodín debe permanecer vacío.

Interfaz

Carta(): Carta. Crea una carta comodín.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene una carta de tipo comodín sin territorio asociado.

Carta(nuevoTipo, nuevoCodigoTerritorio): Carta. Crea una carta con los datos recibidos.

- Precondiciones: nuevoTipo debe corresponder con un tipo de carta válido. Si es un comodín, el código debe estar vacío. En los demás casos debe identificar un territorio.
- Poscondiciones: Se obtiene una carta con el tipo y el código recibidos.

obtenerTipo(): TipoCarta. Consulta el tipo de la carta.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna tipo sin modificar la carta.

obtenerCodigoTerritorio(): Cadena de caracteres. Consulta el territorio asociado con la carta.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna codigoTerritorio sin modificar la carta.

esComodin(): Booleano. Comprueba si la carta es un comodín.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si tipo corresponde con comodín y falso en caso contrario.

TAD Jugador

Representa a uno de los participantes de la partida y las cartas que ha recibido.

Estado

- Nombre del jugador (nombre), Cadena de caracteres. Identifica al jugador.
- Condición: Al cargar una partida debe contener máximo ocho caracteres y no puede repetirse entre los jugadores.
- Color del jugador (color), Cadena de caracteres. Identifica los territorios controlados por el jugador.
- Condición: Debe corresponder con uno de los colores permitidos y no puede repetirse dentro de la partida.
- Cartas del jugador (cartas), Secuencia de Carta. Contiene las cartas obtenidas durante la partida.
- Condición: Cada carta debe corresponder con una carta que haya sido retirada del mazo y entregada al jugador.

Interfaz

Jugador(): Jugador. Crea un jugador vacío.
- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un jugador con nombre y color vacíos y sin cartas.

Jugador(nuevoNombre, nuevoColor): Jugador. Crea un jugador identificado.
- Precondiciones: nuevoNombre y nuevoColor deben cumplir las condiciones definidas para los jugadores.
- Poscondiciones: Se obtiene un jugador con el nombre y color recibidos. La secuencia de cartas queda vacía.

obtenerNombre(): Cadena de caracteres. Consulta el nombre del jugador.
- Precondiciones: Ninguna.
- Poscondiciones: Retorna nombre sin modificar el jugador.

obtenerColor(): Cadena de caracteres. Consulta el color del jugador.
- Precondiciones: Ninguna.
- Poscondiciones: Retorna color sin modificar el jugador.

agregarCarta(nuevaCarta): Vacío. Agrega una carta al jugador.
- Precondiciones: nuevaCarta debe ser una carta válida.
- Poscondiciones: La carta recibida queda agregada al final de cartas.

obtenerCartas(): Secuencia de Carta. Permite consultar las cartas del jugador.
- Precondiciones: Ninguna.
- Poscondiciones: Retorna la secuencia cartas. La operación de consulta no cambia el jugador.

TAD Mazo

Administra las cartas que todavía pueden entregarse y la cantidad de intercambios realizados.

Estado
- Cartas disponibles (cartasDisponibles), Secuencia de Carta. Contiene las cartas que aún no han sido entregadas.
- Condición: Después de inicializar el mazo puede contener como máximo 44 cartas, correspondientes a 42 territorios y dos comodines.
- Cantidad de intercambios (cantidadIntercambios), Numérico. Cuenta los intercambios realizados por todos los jugadores.
- Condición: cantidadIntercambios ≥ 0.

Interfaz

Mazo(): Mazo. Crea un mazo vacío.
- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un mazo sin cartas disponibles y con cero intercambios.

inicializarMazo(): Vacío. Crea las cartas utilizadas durante la partida.
- Precondiciones: Ninguna.
- Poscondiciones: cartasDisponibles queda formada por 42 cartas de territorio y dos comodines. cantidadIntercambios queda en cero.

tieneCartasDisponibles(): Booleano. Comprueba si el mazo todavía puede entregar una carta.
- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si cartasDisponibles contiene al menos una carta y falso si está vacía.

entregarCarta(jugador): Booleano. Intenta entregar una carta al jugador.
- Precondiciones: Ninguna.
- Poscondiciones: Si existen cartas disponibles, selecciona una, la agrega al jugador, la elimina del mazo y retorna verdadero. Si el mazo está vacío, retorna falso y conserva su estado.

combinacionValida(cartas): Booleano. Comprueba si tres cartas forman una combinación permitida.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero cuando la secuencia contiene exactamente tres cartas del mismo tipo, una de cada tipo de ejército o dos cartas de ejército y un comodín. En los demás casos retorna falso. Las cartas no se modifican.

procesarIntercambio(jugador, tablero, posicionUno, posicionDos, posicionTres): Numérico. Valida y realiza el intercambio de tres cartas.

- Precondiciones: Ninguna.
- Poscondiciones: Si las posiciones existen, son diferentes y forman una combinación válida, aumenta cantidadIntercambios, calcula la bonificación, agrega dos unidades en los territorios propios correspondientes y elimina las tres cartas del jugador. Retorna la bonificación obtenida. Si la selección no es válida, retorna cero y conserva el estado anterior.

TAD Dados

Guarda los resultados de los dados utilizados durante una ronda de ataque.

Estado

- Resultados (resultados), Secuencia de Numéricos. Contiene los valores obtenidos en el último lanzamiento.
- Condición: Cada resultado debe encontrarse entre 1 y 6. Después de lanzar los dados, los valores deben permanecer ordenados de mayor a menor.

Interfaz

Dados(): Dados. Crea un conjunto de dados sin resultados almacenados.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un conjunto de dados con resultados vacío.

lanzar(cantidadDados): Vacío. Genera y ordena la cantidad solicitada de resultados.

- Precondiciones: Ninguna.
- Poscondiciones: Elimina los resultados anteriores. Si cantidadDados > 0, genera esa cantidad de valores entre 1 y 6 y los ordena de mayor a menor. Si cantidadDados ≤ 0, la secuencia queda vacía.

obtenerResultados(): Secuencia de Numéricos. Consulta todos los resultados del último lanzamiento.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna resultados sin modificar los dados.

obtenerMayores(cantidad): Secuencia de Numéricos. Consulta los mayores resultados disponibles.

- Precondiciones: Ninguna.
- Poscondiciones: Si cantidad > 0, retorna los primeros resultados ordenados sin superar la cantidad disponible. Si cantidad ≤ 0, retorna una secuencia vacía. Los dados no se modifican.

limpiar(): Vacío. Elimina los resultados almacenados.

- Precondiciones: Ninguna.
- Poscondiciones: resultados queda vacío.

TAD Ataque

Controla la selección de los territorios y las rondas de dados entre el atacante y el defensor.

Estado

- Dados del atacante (dadosAtacante), Dados. Conserva los resultados del último lanzamiento del atacante.
- Condición: Durante una ronda válida contiene entre uno y tres resultados.
- Dados del defensor (dadosDefensor), Dados. Conserva los resultados del último lanzamiento del defensor.
- Condición: Durante una ronda válida contiene uno o dos resultados.

Interfaz

Ataque(): Ataque. Crea el controlador de una etapa de ataque.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un ataque con los dados del atacante y del defensor sin resultados almacenados.

configuracionValida(codigoOrigen, codigoDestino, colorAtacante, tablero): Booleano. Comprueba si los territorios seleccionados permiten realizar un ataque.

- Precondiciones: El tablero debe contener los territorios del juego.
- Poscondiciones: Retorna verdadero si ambos territorios existen, son vecinos, el origen pertenece al atacante, el destino pertenece a otro jugador y el origen tiene más de una unidad. En caso contrario, retorna falso. El tablero no se modifica.

ejecutar(jugadorAtacante, tablero): ResultadoAtaque. Solicita los territorios, realiza las rondas de dados y registra una conquista cuando corresponde.

- Precondiciones: El jugador debe encontrarse en la etapa de ataque y el tablero debe estar inicializado.
- Poscondiciones: Retorna un ResultadoAtaque que indica si se realizó al menos una ronda y si se conquistó un territorio. Puede modificar las unidades de los territorios y el propietario del territorio conquistado.

TAD Turno

Conserva la posición del jugador actual y la etapa que puede ejecutar.

Estado

- Posición del jugador actual (posicionJugadorActual), Numérico. Indica la posición del jugador que tiene el turno.
- Condición: $0 \leq$ posicionJugadorActual $<$ cantidad de jugadores.
- Etapa actual (etapaActual), EtapaTurno. Indica la acción que puede ejecutar el jugador.
- Condición: Debe corresponder con obtención de unidades, ataque o fortificación.

Interfaz

Turno(): Turno. Crea el control inicial del turno.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un turno con la posición del jugador actual igual a cero y la etapa de obtención de unidades.

iniciar(primeraPosicion): Vacío. Establece la posición del primer jugador.

- Precondiciones: primeraPosicion debe corresponder con una posición existente dentro de la secuencia de jugadores.
- Poscondiciones: posicionJugadorActual queda igual a la posición recibida y etapaActual queda en obtención de unidades.

obtenerPosicionJugadorActual(): Numérico. Consulta la posición del jugador actual.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna posicionJugadorActual sin modificar el turno.

obtenerEtapaActual(): EtapaTurno. Consulta la etapa actual.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna etapaActual sin modificar el turno.

esTurnoDe(posicionJugador): Booleano. Comprueba si una posición corresponde con el jugador actual.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si posicionJugador coincide con posicionJugadorActual. En caso contrario, retorna falso.

puedeObtenerUnidades(): Booleano. Comprueba si el turno se encuentra en la etapa de obtención.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero únicamente cuando etapaActual corresponde con obtención de unidades.

puedeAtacar(): Booleano. Comprueba si el turno se encuentra en la etapa de ataque.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero únicamente cuando etapaActual corresponde con ataque.

puedeFortificar(): Booleano. Comprueba si el turno se encuentra en la etapa de fortificación.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero únicamente cuando etapaActual corresponde con fortificación.

calcularNuevasUnidades(jugador, tablero, bonificacionCartas): Numérico. Calcula las unidades que obtiene un jugador.

- Precondiciones: bonificacionCartas ≥ 0.

- Poscondiciones: Retorna la división entera de los territorios controlados entre tres, más la bonificación por continentes y la bonificación por cartas. El jugador, el tablero y el turno conservan su estado.

ejecutarObtencionUnidades(jugador, tablero, codigoTerritorio, cantidad): Booleano. Intenta agregar una parte de las unidades obtenidas.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es obtención, cantidad > 0 y el territorio pertenece al jugador, agrega las unidades y retorna verdadero. En caso contrario, retorna falso y conserva el tablero.

registrarUnidadesObtenidas(): Vacío. Registra el final de la etapa de obtención.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es obtención, cambia a ataque. En cualquier otra etapa conserva su valor.

registrarAtaqueTerminado(): Vacío. Registra el final de la etapa de ataque.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es ataque, cambia a fortificación. En cualquier otra etapa conserva su valor.

ejecutarFortificacion(jugador, tablero, codigoOrigen, codigoDestino, cantidad): Booleano. Intenta trasladar unidades durante la fortificación.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es fortificación, la cantidad es positiva y el traslado es válido, modifica el tablero y retorna verdadero. En caso contrario, retorna falso y conserva el tablero.

finalizarTurno(cantidadJugadores): Vacío. Avanza al siguiente jugador.

- Precondiciones: cantidadJugadores > 0.
- Poscondiciones: Aumenta circularmente posicionJugadorActual. Después de la última posición vuelve a cero y establece la etapa de obtención de unidades.

TAD Partida

Integra los elementos del juego y controla la ejecución de los comandos del Componente 1.

Estado

- Jugadores (jugadores), Secuencia de Jugador. Contiene los participantes de la partida.
- Condición: Antes de inicializar puede estar vacía. Después de cargar una partida válida debe contener entre tres y seis jugadores, sin nombres ni colores repetidos.
- Tablero (tablero), Tablero. Conserva los continentes, territorios, propietarios y unidades.
- Condición: Después de inicializar la partida debe contener los seis continentes y los 42 territorios configurados.
- Turno (turno), Turno. Controla el jugador actual y la etapa que puede ejecutar.
- Condición: La posición actual debe identificar a uno de los jugadores que todavía conserva territorios.
- Ataque (ataque), Ataque. Controla las rondas de combate.
- Condición: Sus dados deben cumplir las condiciones establecidas por el TAD Dados.
- Mazo (mazo), Mazo. Conserva las cartas disponibles y la cantidad de intercambios.
- Condición: Después de inicializar la partida debe contener las cartas que todavía no se han entregado.
- Partida inicializada (inicializada), Booleano. Indica si el archivo inicial se cargó correctamente.
- Condición: Solo puede ser verdadero cuando los jugadores y los 42 territorios fueron validados y almacenados.
- Partida terminada (terminada), Booleano. Indica si ya existe un ganador.
- Condición: Solo puede ser verdadero cuando un jugador controla los 42 territorios.
- Nombre del ganador (ganador), Cadena de caracteres. Guarda el nombre del jugador que ganó.
- Condición: Permanece vacío mientras terminada sea falso. Cuando terminada sea verdadero debe corresponder con uno de los jugadores.

Interfaz

Partida(): Partida. Crea una partida sin información cargada.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene una partida con inicializada y terminada en falso y con ganador vacío.

inicializar(nombreArchivo): EstadoComando. Intenta cargar la información inicial desde un archivo.

- Precondiciones: Ninguna.

- Poscondiciones: Si no existe otra partida activa y el archivo cumple el formato, guarda los jugadores, configura el tablero, inicia el turno, prepara el mazo, establece inicializada en verdadero y retorna COMANDO_CORRECTO. Si ocurre un error, retorna el estado correspondiente y no conserva una partida incompleta.

obtenerUnidades(nombreJugador): EstadoComando. Gestiona los intercambios de cartas y la distribución de unidades.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está activa, el jugador existe, tiene el turno y se encuentra en la etapa de obtención, permite realizar canjes y ubicar todas las unidades en territorios propios. Al completar el proceso guarda los cambios, avanza a ataque y retorna COMANDO_CORRECTO. Si las condiciones no se cumplen, retorna el estado correspondiente.

atacar(nombreJugador): EstadoComando. Controla la etapa de ataque del jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está activa, el jugador tiene el turno y se encuentra en la etapa de ataque, permite realizar ataques. Puede modificar territorios, entregar máximo una carta y registrar al ganador. Si el jugador termina la etapa sin ganar, el turno avanza a fortificación.

fortificar(nombreJugador): EstadoComando. Realiza el traslado opcional y termina el turno.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está activa y el jugador se encuentra en la etapa de fortificación, permite realizar un traslado válido o terminar sin trasladar. Después avanza al siguiente jugador que conserve territorios y establece la etapa de obtención.

validarEstadoJuego(): EstadoComando. Comprueba si la partida puede utilizarse.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna JUEGO_NO_INICIALIZADO si la partida no ha sido cargada, JUEGO_TERMINADO si ya existe un ganador o COMANDO_CORRECTO si la partida continúa. El estado no se modifica.

buscarPosicionJugador(nombreJugador): Numérico. Busca la posición de un jugador mediante su nombre.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la posición del jugador si lo encuentra. Si no existe, retorna -1. La secuencia de jugadores no se modifica.

comprobarGanador(): Booleano. Comprueba si algún jugador controla los 42 territorios.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está inicializada y encuentra un jugador con 42 territorios, establece terminada en verdadero, guarda su nombre en ganador y retorna verdadero. En caso contrario, retorna falso.

mostrarEstadoJuego(): Vacío. Presenta los jugadores, el turno actual y el tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está inicializada y no ha terminado, muestra la información actual. La partida conserva su estado.

Descripción de los Tipos Abstractos de Datos

Los Tipos Abstractos de Datos se describen mediante el conjunto mínimo de datos que forma su estado y las operaciones que componen su interfaz. Para cada variable se indican sus condiciones cuando corresponde. Cada operación presenta sus argumentos, el valor que retorna, su función, las precondiciones y las poscondiciones, siguiendo la plantilla trabajada en clase [2].

TAD Territorio

Representa un territorio del tablero, junto con su identificación, propietario, cantidad de unidades y conexiones con otros territorios.

Estado

- Código del territorio (codigo), Cadena de caracteres. Identifica el territorio.
- Condición: Puede estar vacío únicamente mientras el territorio no haya sido configurado. En un territorio del tablero debe corresponder con uno de los 42 códigos definidos.
- Nombre del territorio (nombre), Cadena de caracteres. Guarda el nombre con el que se presenta el territorio.

- Condición: Puede estar vacío únicamente mientras el territorio no haya sido configurado.
- Código del continente (codigoContinente), Cadena de caracteres. Identifica el continente al que pertenece el territorio.
- Condición: En un territorio configurado debe corresponder con uno de los seis continentes del tablero.
- Color del propietario (colorPropietario), Cadena de caracteres. Identifica al jugador que controla el territorio.
- Condición: Puede estar vacío antes de cargar la partida. Después de configurar el tablero debe corresponder con el color de uno de los jugadores.
- Cantidad de unidades (unidades), Numérico. Indica cuántas unidades se encuentran en el territorio.
- Condición: unidades ≥ 0.
- Códigos de los vecinos (codigosVecinos), Secuencia de Cadenas de caracteres. Guarda los códigos de los territorios conectados directamente.
- Condición: Cada código debe identificar un territorio vecino definido en el tablero.

Interfaz

Territorio(): Territorio. Crea un territorio vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un territorio con sus datos de identificación y propietario vacíos, cero unidades y sin vecinos registrados.

Territorio(nuevoCodigo, nuevoNombre, nuevoContinente): Territorio. Crea un territorio con los datos de identificación recibidos.

- Precondiciones: nuevoCodigo, nuevoNombre y nuevoContinente deben representar un territorio definido para el tablero.
- Poscondiciones: Se obtiene un territorio con el código, nombre y continente recibidos. El propietario queda vacío, la cantidad de unidades queda en cero y no se registran vecinos.

obtenerCodigo(): Cadena de caracteres. Consulta el código del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de codigo sin modificar el territorio.

obtenerNombre(): Cadena de caracteres. Consulta el nombre del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de nombre sin modificar el territorio.

obtenerCodigoContinente(): Cadena de caracteres. Consulta el código del continente al que pertenece el territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de codigoContinente sin modificar el territorio.

obtenerColorPropietario(): Cadena de caracteres. Consulta el color del propietario.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de colorPropietario sin modificar el territorio.

obtenerUnidades(): Numérico. Consulta la cantidad de unidades del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de unidades sin modificar el territorio.

asignarPropietario(nuevoColor): Vacío. Asigna directamente el propietario del territorio.

- Precondiciones: nuevoColor debe corresponder con el color que se desea cargar.
- Poscondiciones: colorPropietario queda igual a nuevoColor. Los demás datos conservan su valor.

asignarUnidades(nuevaCantidad): Vacío. Asigna directamente la cantidad de unidades del territorio.

- Precondiciones: nuevaCantidad ≥ 0.
- Poscondiciones: unidades queda igual a nuevaCantidad. Los demás datos conservan su valor.

agregarUnidades(cantidad): Booleano. Intenta agregar unidades al territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si cantidad > 0, aumenta unidades en la cantidad recibida y retorna verdadero. Si cantidad ≤ 0, retorna falso y conserva el estado anterior.

retirarUnidades(cantidad, minimoRestante): Booleano. Intenta retirar unidades sin superar el mínimo que debe permanecer.

- Precondiciones: Ninguna.
- Poscondiciones: Si cantidad > 0 y unidades - cantidad ≥ minimoRestante, descuenta la cantidad y retorna verdadero. En caso contrario, retorna falso y conserva la cantidad de unidades.

agregarVecino(codigoVecino): Vacío. Registra un territorio como vecino.

- Precondiciones: codigoVecino debe identificar un territorio conectado directamente.
- Poscondiciones: El código recibido queda agregado al final de codigosVecinos.

esVecino(codigoTerritorio): Booleano. Comprueba si un territorio se encuentra registrado como vecino.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si codigoTerritorio se encuentra en codigosVecinos. En caso contrario, retorna falso. El territorio no se modifica.

obtenerVecinos(): Secuencia de Cadenas de caracteres. Consulta todos los códigos de los territorios vecinos.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la secuencia codigosVecinos sin modificarla.

cambiarPropietario(color, cantidad): Booleano. Registra la conquista del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio tiene cero unidades y cantidad > 0, cambia colorPropietario, agrega la cantidad recibida y retorna verdadero. En caso contrario, retorna falso y conserva el estado anterior.

TAD Continente

Representa una región del tablero que reúne varios territorios y entrega una bonificación cuando un jugador la controla completamente.

Estado

- Código del continente (codigo), Cadena de caracteres. Identifica el continente.
- Condición: Puede estar vacío únicamente mientras el continente no haya sido configurado. En el tablero debe corresponder con uno de los seis códigos definidos.
- Nombre del continente (nombre), Cadena de caracteres. Guarda el nombre visible del continente.
- Condición: Puede estar vacío únicamente mientras el continente no haya sido configurado.
- Bonificación (bonificacion), Numérico. Indica la cantidad de unidades adicionales que entrega el continente.
- Condición: En un continente configurado debe corresponder con la bonificación definida en el enunciado. En un continente vacío su valor es cero.
- Territorios (territorios), Secuencia de Territorio. Contiene los territorios que forman el continente.
- Condición: Cada territorio debe tener un código de continente igual al código del continente que lo contiene.

Interfaz

Continente(): Continente. Crea un continente vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un continente con código y nombre vacíos, bonificación igual a cero y sin territorios.

Continente(nuevoCodigo, nuevoNombre, nuevaBonificacion): Continente. Crea un continente identificado.

- Precondiciones: Los datos deben corresponder con uno de los continentes definidos en el tablero.
- Poscondiciones: Se obtiene un continente con el código, nombre y bonificación recibidos. La secuencia de territorios queda vacía.

obtenerCodigo(): Cadena de caracteres. Consulta el código del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna codigo sin modificar el continente.

obtenerNombre(): Cadena de caracteres. Consulta el nombre del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna nombre sin modificar el continente.

obtenerBonificacion(): Numérico. Consulta la bonificación del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna bonificacion sin modificar el continente.

agregarTerritorio(territorio): Vacío. Agrega un territorio al continente.

- Precondiciones: El código de continente del territorio debe corresponder con el continente que lo recibe.
- Poscondiciones: El territorio queda agregado al final de territorios.

obtenerTerritorios(): Secuencia de Territorio. Consulta los territorios del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la secuencia territorios sin modificarla.

estaControladoPor(colorJugador): Booleano. Comprueba si todos los territorios pertenecen al jugador indicado.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si la secuencia no está vacía y todos sus territorios tienen como propietario a colorJugador. En caso contrario, retorna falso.

buscarTerritorio(codigo): Territorio o inexistente. Busca un territorio dentro del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Si encuentra un territorio con el código recibido, retorna ese territorio. Si no lo encuentra, informa que no existe.

TAD Tablero

Representa el mapa completo de Risk mediante los continentes y territorios que participan en la partida.

Estado

- Continentes (continentes), Secuencia de Continente. Contiene las regiones que forman el tablero.
- Condición: Antes de inicializar el tablero puede estar vacía. Después de ejecutar inicializarEstructura, debe contener seis continentes y un total de 42 territorios.

Interfaz

Tablero(): Tablero. Crea un tablero vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un tablero sin continentes.

inicializarEstructura(): Vacío. Construye los continentes, territorios y conexiones del mapa.

- Precondiciones: Ninguna.
- Poscondiciones: Reemplaza el estado anterior por los seis continentes y los 42 territorios definidos para el juego.

buscarTerritorio(codigo): Territorio o inexistente. Busca un territorio en todo el tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Si encuentra un territorio con el código recibido, retorna ese territorio. Si no lo encuentra, informa que no existe.

existeTerritorio(codigo): Booleano. Comprueba si un código identifica un territorio del tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si encuentra el territorio y falso en caso contrario. El tablero conserva su estado.

configurarTerritorio(codigo, colorPropietario, unidades): Booleano. Carga el propietario y las unidades iniciales de un territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio existe, asigna el color y las unidades recibidas y retorna verdadero. Si no existe, retorna falso y conserva el tablero.

sonVecinos(codigoOrigen, codigoDestino): Booleano. Comprueba si dos territorios tienen una conexión directa.

- Precondiciones: Ninguna.
- Poscondiciones: Si ambos territorios existen, retorna el resultado de buscar codigoDestino entre los vecinos de codigoOrigen. Si alguno no existe, retorna falso. El tablero no se modifica.

contarTerritorios(colorJugador): Numérico. Cuenta los territorios pertenecientes al jugador indicado.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la cantidad de territorios cuyo propietario corresponde con colorJugador. El tablero no se modifica.

sumarUnidades(colorJugador): Numérico. Suma las unidades de todos los territorios pertenecientes a un jugador.

- Precondiciones: Ninguna.

- Poscondiciones: Retorna la suma de unidades de los territorios cuyo propietario corresponde con colorJugador. El tablero no se modifica.

calcularBonificacionContinentes(colorJugador): Numérico. Calcula la bonificación por continentes controlados.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la suma de las bonificaciones de los continentes controlados completamente por colorJugador. El tablero no se modifica.

agregarUnidades(codigoTerritorio, colorJugador, cantidad): Booleano. Intenta agregar unidades a un territorio propio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio existe, pertenece a colorJugador y cantidad > 0, agrega las unidades y retorna verdadero. En caso contrario, retorna falso y conserva el estado anterior.

trasladarUnidades(codigoOrigen, codigoDestino, colorJugador, cantidad, minimoRestante): Booleano. Intenta trasladar unidades entre dos territorios propios vecinos.

- Precondiciones: Ninguna.
- Poscondiciones: Si ambos territorios existen, son vecinos, pertenecen a colorJugador y el origen puede conservar el mínimo indicado, descuenta la cantidad del origen, la agrega al destino y retorna verdadero. En caso contrario, retorna falso y conserva las unidades de ambos territorios.

cambiarPropietario(codigoTerritorio, nuevoColor, cantidad): Booleano. Intenta registrar la conquista de un territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio existe, pertenece a otro jugador, tiene cero unidades y la cantidad recibida es positiva, cambia su propietario, asigna las nuevas unidades y retorna verdadero. En caso contrario, retorna falso.

todosLosTerritoriosConfigurados(): Booleano. Comprueba si todos los territorios tienen propietario.

- Precondiciones: El tablero debe contener los continentes y territorios del juego.
- Poscondiciones: Retorna verdadero si ningún territorio tiene el propietario vacío. Si encuentra al menos uno sin propietario, retorna falso. El tablero no se modifica.

mostrarTablero(): Vacío. Presenta los continentes y territorios del tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Muestra el nombre, código, propietario y unidades de cada territorio. El tablero conserva su estado.

TAD Carta

Representa una carta de territorio o un comodín utilizado durante los intercambios.

Estado

- Tipo de carta (tipo), TipoCarta. Identifica la figura de la carta.
- Condición: Debe corresponder con infantería, caballería, artillería o comodín.
- Código del territorio (codigoTerritorio), Cadena de caracteres. Identifica el territorio asociado con la carta.
- Condición: Debe corresponder con un territorio del tablero cuando la carta no es un comodín. Para un comodín debe permanecer vacío.

Interfaz

Carta(): Carta. Crea una carta comodín.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene una carta de tipo comodín sin territorio asociado.

Carta(nuevoTipo, nuevoCodigoTerritorio): Carta. Crea una carta con los datos recibidos.

- Precondiciones: nuevoTipo debe corresponder con un tipo de carta válido. Si es un comodín, el código debe estar vacío. En los demás casos debe identificar un territorio.
- Poscondiciones: Se obtiene una carta con el tipo y el código recibidos.

obtenerTipo(): TipoCarta. Consulta el tipo de la carta.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna tipo sin modificar la carta.

obtenerCodigoTerritorio(): Cadena de caracteres. Consulta el territorio asociado con la carta.

- Precondiciones: Ninguna.

- Poscondiciones: Retorna codigoTerritorio sin modificar la carta.

esComodin(): Booleano. Comprueba si la carta es un comodín.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si tipo corresponde con comodín y falso en caso contrario.

TAD Jugador

Representa a uno de los participantes de la partida y las cartas que ha recibido.

Estado

- Nombre del jugador (nombre), Cadena de caracteres. Identifica al jugador.
- Condición: Al cargar una partida debe contener máximo ocho caracteres y no puede repetirse entre los jugadores.
- Color del jugador (color), Cadena de caracteres. Identifica los territorios controlados por el jugador.
- Condición: Debe corresponder con uno de los colores permitidos y no puede repetirse dentro de la partida.
- Cartas del jugador (cartas), Secuencia de Carta. Contiene las cartas obtenidas durante la partida.
- Condición: Cada carta debe corresponder con una carta que haya sido retirada del mazo y entregada al jugador.

Interfaz

Jugador(): Jugador. Crea un jugador vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un jugador con nombre y color vacíos y sin cartas.

Jugador(nuevoNombre, nuevoColor): Jugador. Crea un jugador identificado.

- Precondiciones: nuevoNombre y nuevoColor deben cumplir las condiciones definidas para los jugadores.
- Poscondiciones: Se obtiene un jugador con el nombre y color recibidos. La secuencia de cartas queda vacía.

obtenerNombre(): Cadena de caracteres. Consulta el nombre del jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna nombre sin modificar el jugador.

obtenerColor(): Cadena de caracteres. Consulta el color del jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna color sin modificar el jugador.

agregarCarta(nuevaCarta): Vacío. Agrega una carta al jugador.

- Precondiciones: nuevaCarta debe ser una carta válida.
- Poscondiciones: La carta recibida queda agregada al final de cartas.

obtenerCartas(): Secuencia de Carta. Permite consultar las cartas del jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la secuencia cartas. La operación de consulta no cambia el jugador.

TAD Mazo

Administra las cartas que todavía pueden entregarse y la cantidad de intercambios realizados.

Estado

- Cartas disponibles (cartasDisponibles), Secuencia de Carta. Contiene las cartas que aún no han sido entregadas.
- Condición: Después de inicializar el mazo puede contener como máximo 44 cartas, correspondientes a 42 territorios y dos comodines.
- Cantidad de intercambios (cantidadIntercambios), Numérico. Cuenta los intercambios realizados por todos los jugadores.
- Condición: cantidadIntercambios ≥ 0.

Interfaz

Mazo(): Mazo. Crea un mazo vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un mazo sin cartas disponibles y con cero intercambios.

inicializarMazo(): Vacío. Crea las cartas utilizadas durante la partida.

- Precondiciones: Ninguna.

- Poscondiciones: cartasDisponibles queda formada por 42 cartas de territorio y dos comodines. cantidadIntercambios queda en cero.

tieneCartasDisponibles(): Booleano. Comprueba si el mazo todavía puede entregar una carta.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si cartasDisponibles contiene al menos una carta y falso si está vacía.

entregarCarta(jugador): Booleano. Intenta entregar una carta al jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Si existen cartas disponibles, selecciona una, la agrega al jugador, la elimina del mazo y retorna verdadero. Si el mazo está vacío, retorna falso y conserva su estado.

combinacionValida(cartas): Booleano. Comprueba si tres cartas forman una combinación permitida.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero cuando la secuencia contiene exactamente tres cartas del mismo tipo, una de cada tipo de ejército o dos cartas de ejército y un comodín. En los demás casos retorna falso. Las cartas no se modifican.

procesarIntercambio(jugador, tablero, posicionUno, posicionDos, posicionTres): Numérico. Valida y realiza el intercambio de tres cartas.

- Precondiciones: Ninguna.
- Poscondiciones: Si las posiciones existen, son diferentes y forman una combinación válida, aumenta cantidadIntercambios, calcula la bonificación, agrega dos unidades en los territorios propios correspondientes y elimina las tres cartas del jugador. Retorna la bonificación obtenida. Si la selección no es válida, retorna cero y conserva el estado anterior.

TAD Dados

Guarda los resultados de los dados utilizados durante una ronda de ataque.

Estado

- Resultados (resultados), Secuencia de Numéricos. Contiene los valores obtenidos en el último lanzamiento.
- Condición: Cada resultado debe encontrarse entre 1 y 6. Después de lanzar los dados, los valores deben permanecer ordenados de mayor a menor.

Interfaz

Dados(): Dados. Crea un conjunto de dados sin resultados almacenados.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un conjunto de dados con resultados vacío.

lanzar(cantidadDados): Vacío. Genera y ordena la cantidad solicitada de resultados.

- Precondiciones: Ninguna.
- Poscondiciones: Elimina los resultados anteriores. Si cantidadDados > 0, genera esa cantidad de valores entre 1 y 6 y los ordena de mayor a menor. Si cantidadDados ≤ 0, la secuencia queda vacía.

obtenerResultados(): Secuencia de Numéricos. Consulta todos los resultados del último lanzamiento.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna resultados sin modificar los dados.

obtenerMayores(cantidad): Secuencia de Numéricos. Consulta los mayores resultados disponibles.

- Precondiciones: Ninguna.
- Poscondiciones: Si cantidad > 0, retorna los primeros resultados ordenados sin superar la cantidad disponible. Si cantidad ≤ 0, retorna una secuencia vacía. Los dados no se modifican.

limpiar(): Vacío. Elimina los resultados almacenados.

- Precondiciones: Ninguna.
- Poscondiciones: resultados queda vacío.

TAD Ataque

Controla la selección de los territorios y las rondas de dados entre el atacante y el defensor.

Estado

- Dados del atacante (dadosAtacante), Dados. Conserva los resultados del último lanzamiento del atacante.
- Condición: Durante una ronda válida contiene entre uno y tres resultados.
- Dados del defensor (dadosDefensor), Dados. Conserva los resultados del último lanzamiento del defensor.
- Condición: Durante una ronda válida contiene uno o dos resultados.

Interfaz

Ataque(): Ataque. Crea el controlador de una etapa de ataque.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un ataque con los dados del atacante y del defensor sin resultados almacenados.

configuracionValida(codigoOrigen, codigoDestino, colorAtacante, tablero): Booleano. Comprueba si los territorios seleccionados permiten realizar un ataque.

- Precondiciones: El tablero debe contener los territorios del juego.
- Poscondiciones: Retorna verdadero si ambos territorios existen, son vecinos, el origen pertenece al atacante, el destino pertenece a otro jugador y el origen tiene más de una unidad. En caso contrario, retorna falso. El tablero no se modifica.

ejecutar(jugadorAtacante, tablero): ResultadoAtaque. Solicita los territorios, realiza las rondas de dados y registra una conquista cuando corresponde.

- Precondiciones: El jugador debe encontrarse en la etapa de ataque y el tablero debe estar inicializado.
- Poscondiciones: Retorna un ResultadoAtaque que indica si se realizó al menos una ronda y si se conquistó un territorio. Puede modificar las unidades de los territorios y el propietario del territorio conquistado.

TAD Turno

Conserva la posición del jugador actual y la etapa que puede ejecutar.

Estado

- Posición del jugador actual (posicionJugadorActual), Numérico. Indica la posición del jugador que tiene el turno.
- Condición: $0 \leq$ posicionJugadorActual $<$ cantidad de jugadores.
- Etapa actual (etapaActual), EtapaTurno. Indica la acción que puede ejecutar el jugador.
- Condición: Debe corresponder con obtención de unidades, ataque o fortificación.

Interfaz

Turno(): Turno. Crea el control inicial del turno.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un turno con la posición del jugador actual igual a cero y la etapa de obtención de unidades.

iniciar(primeraPosicion): Vacío. Establece la posición del primer jugador.

- Precondiciones: primeraPosicion debe corresponder con una posición existente dentro de la secuencia de jugadores.
- Poscondiciones: posicionJugadorActual queda igual a la posición recibida y etapaActual queda en obtención de unidades.

obtenerPosicionJugadorActual(): Numérico. Consulta la posición del jugador actual.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna posicionJugadorActual sin modificar el turno.

obtenerEtapaActual(): EtapaTurno. Consulta la etapa actual.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna etapaActual sin modificar el turno.

esTurnoDe(posicionJugador): Booleano. Comprueba si una posición corresponde con el jugador actual.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si posicionJugador coincide con posicionJugadorActual. En caso contrario, retorna falso.

puedeObtenerUnidades(): Booleano. Comprueba si el turno se encuentra en la etapa de obtención.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero únicamente cuando etapaActual corresponde con obtención de unidades.

puedeAtacar(): Booleano. Comprueba si el turno se encuentra en la etapa de ataque.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero únicamente cuando etapaActual corresponde con ataque.

puedeFortificar(): Booleano. Comprueba si el turno se encuentra en la etapa de fortificación.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero únicamente cuando etapaActual corresponde con fortificación.

calcularNuevasUnidades(jugador, tablero, bonificacionCartas): Numérico. Calcula las unidades que obtiene un jugador.

- Precondiciones: bonificacionCartas ≥ 0.
- Poscondiciones: Retorna la división entera de los territorios controlados entre tres, más la bonificación por continentes y la bonificación por cartas. El jugador, el tablero y el turno conservan su estado.

ejecutarObtencionUnidades(jugador, tablero, codigoTerritorio, cantidad): Booleano. Intenta agregar una parte de las unidades obtenidas.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es obtención, cantidad > 0 y el territorio pertenece al jugador, agrega las unidades y retorna verdadero. En caso contrario, retorna falso y conserva el tablero.

registrarUnidadesObtenidas(): Vacío. Registra el final de la etapa de obtención.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es obtención, cambia a ataque. En cualquier otra etapa conserva su valor.

registrarAtaqueTerminado(): Vacío. Registra el final de la etapa de ataque.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es ataque, cambia a fortificación. En cualquier otra etapa conserva su valor.

ejecutarFortificacion(jugador, tablero, codigoOrigen, codigoDestino, cantidad): Booleano. Intenta trasladar unidades durante la fortificación.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es fortificación, la cantidad es positiva y el traslado es válido, modifica el tablero y retorna verdadero. En caso contrario, retorna falso y conserva el tablero.

finalizarTurno(cantidadJugadores): Vacío. Avanza al siguiente jugador.

- Precondiciones: cantidadJugadores > 0.
- Poscondiciones: Aumenta circularmente posicionJugadorActual. Después de la última posición vuelve a cero y establece la etapa de obtención de unidades.

TAD Partida

Integra los elementos del juego y controla la ejecución de los comandos del Componente 1.

Estado

- Jugadores (jugadores), Secuencia de Jugador. Contiene los participantes de la partida.
- Condición: Antes de inicializar puede estar vacía. Después de cargar una partida válida debe contener entre tres y seis jugadores, sin nombres ni colores repetidos.
- Tablero (tablero), Tablero. Conserva los continentes, territorios, propietarios y unidades.
- Condición: Después de inicializar la partida debe contener los seis continentes y los 42 territorios configurados.
- Turno (turno), Turno. Controla el jugador actual y la etapa que puede ejecutar.
- Condición: La posición actual debe identificar a uno de los jugadores que todavía conserva territorios.
- Ataque (ataque), Ataque. Controla las rondas de combate.
- Condición: Sus dados deben cumplir las condiciones establecidas por el TAD Dados.
- Mazo (mazo), Mazo. Conserva las cartas disponibles y la cantidad de intercambios.
- Condición: Después de inicializar la partida debe contener las cartas que todavía no se han entregado.
- Partida inicializada (inicializada), Booleano. Indica si el archivo inicial se cargó correctamente.
- Condición: Solo puede ser verdadero cuando los jugadores y los 42 territorios fueron validados y almacenados.
- Partida terminada (terminada), Booleano. Indica si ya existe un ganador.
- Condición: Solo puede ser verdadero cuando un jugador controla los 42 territorios.
- Nombre del ganador (ganador), Cadena de caracteres. Guarda el nombre del jugador que ganó.

- Condición: Permanece vacío mientras terminada sea falso. Cuando terminada sea verdadero debe corresponder con uno de los jugadores.

Interfaz

Partida(): Partida. Crea una partida sin información cargada.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene una partida con inicializada y terminada en falso y con ganador vacío.

inicializar(nombreArchivo): EstadoComando. Intenta cargar la información inicial desde un archivo.

- Precondiciones: Ninguna.
- Poscondiciones: Si no existe otra partida activa y el archivo cumple el formato, guarda los jugadores, configura el tablero, inicia el turno, prepara el mazo, establece inicializada en verdadero y retorna COMANDO_CORRECTO. Si ocurre un error, retorna el estado correspondiente y no conserva una partida incompleta.

obtenerUnidades(nombreJugador): EstadoComando. Gestiona los intercambios de cartas y la distribución de unidades.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está activa, el jugador existe, tiene el turno y se encuentra en la etapa de obtención, permite realizar canjes y ubicar todas las unidades en territorios propios. Al completar el proceso guarda los cambios, avanza a ataque y retorna COMANDO_CORRECTO. Si las condiciones no se cumplen, retorna el estado correspondiente.

atacar(nombreJugador): EstadoComando. Controla la etapa de ataque del jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está activa, el jugador tiene el turno y se encuentra en la etapa de ataque, permite realizar ataques. Puede modificar territorios, entregar máximo una carta y registrar al ganador. Si el jugador termina la etapa sin ganar, el turno avanza a fortificación.

fortificar(nombreJugador): EstadoComando. Realiza el traslado opcional y termina el turno.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está activa y el jugador se encuentra en la etapa de fortificación, permite realizar un traslado válido o terminar sin trasladar. Después avanza al siguiente jugador que conserve territorios y establece la etapa de obtención.

validarEstadoJuego(): EstadoComando. Comprueba si la partida puede utilizarse.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna JUEGO_NO_INICIALIZADO si la partida no ha sido cargada, JUEGO_TERMINADO si ya existe un ganador o COMANDO_CORRECTO si la partida continúa. El estado no se modifica.

buscarPosicionJugador(nombreJugador): Numérico. Busca la posición de un jugador mediante su nombre.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la posición del jugador si lo encuentra. Si no existe, retorna -1. La secuencia de jugadores no se modifica.

comprobarGanador(): Booleano. Comprueba si algún jugador controla los 42 territorios.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está inicializada y encuentra un jugador con 42 territorios, establece terminada en verdadero, guarda su nombre en ganador y retorna verdadero. En caso contrario, retorna falso.

mostrarEstadoJuego(): Vacío. Presenta los jugadores, el turno actual y el tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está inicializada y no ha terminado, muestra la información actual. La partida conserva su estado.

Función	Propósito	Datos de entrada	Resultado	Funciones que utiliza
buscarComando	Busca un nombre en TABLA_COMANDOS.	nombre	Retorna la posición o -1. No modifica la tabla.	size
validarCantidadParametros	Comprueba el mínimo y máximo de argumentos.	posicion, cantidad	Retorna true o false; cuando falla muestra uso y ejemplo.	TABLA_COMANDOS
validarComando	Verifica existencia y cantidad de argumentos.	nombre, cantidadArgumentos	Retorna true si puede ejecutarse; de lo contrario muestra el error.	buscarComando, validarCantidadParametros
separarEntrada	Separa la línea en palabras sin conservar espacios vacíos.	linea	Retorna vector<string> con comando y argumentos.	size, push_back
mostrarAyudaGeneral	Muestra los comandos por categorías.	Ninguno	Imprime usos y ejemplo; no modifica la partida.	TABLA_COMANDOS, size
mostrarAyudaComando	Muestra la ayuda de un comando.	nombre	Imprime descripción, uso, ejemplo y parámetros, o informa que no existe.	buscarComando

Procedimiento principal

Ubicación. La función main está implementada en Entrega0_Risk.cpp.

Entrada. Líneas de texto escritas después del indicador $.

Salida. Indicador de entrada, mensajes de ayuda, validación y resultados de los comandos.

Precondiciones. Ninguna.

Poscondiciones. La aplicación conserva una única partida durante la ejecución, procesa comandos mientras continuar sea verdadero y retorna 0 cuando finaliza.

El flujo inicializa una vez la generación pseudoaleatoria, crea la partida, lee cada línea con getline, separa las palabras, valida el comando y dirige la ejecución a la operación correspondiente.

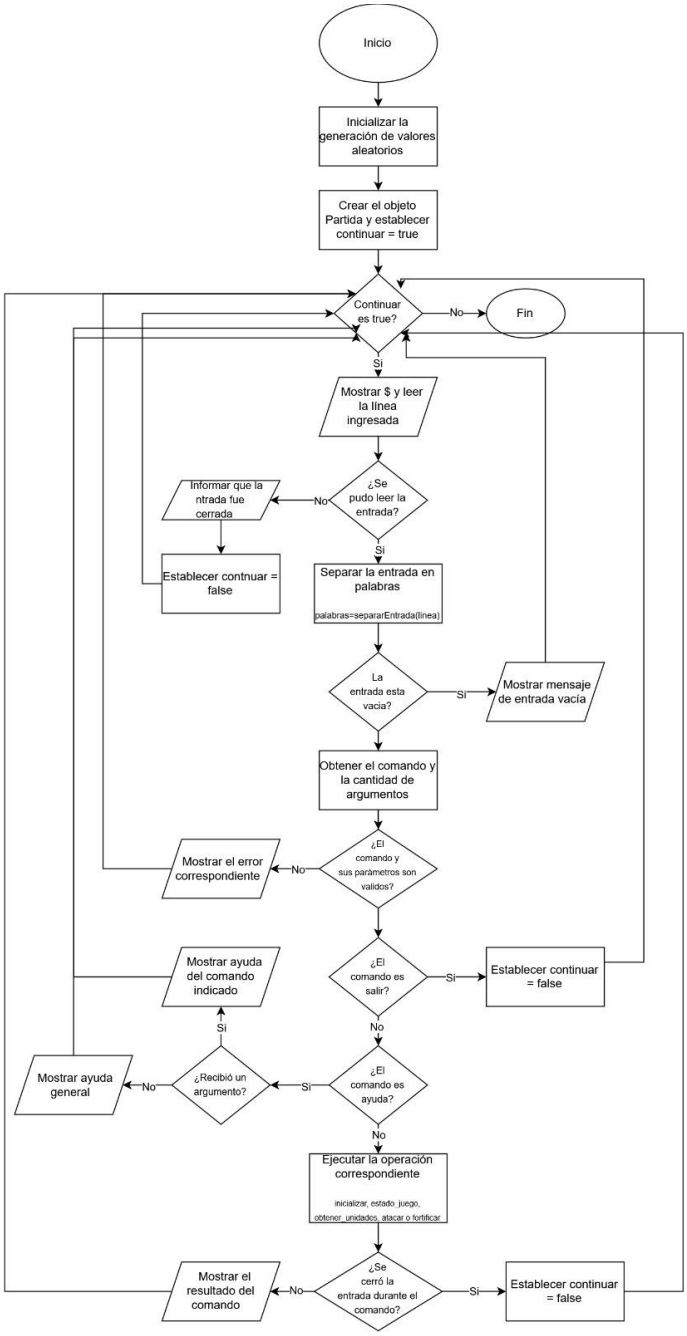


Figura 2. Diagrama de flujo de la función main ubicada en Entrega0_Risk.cpp. Elaboración propia.

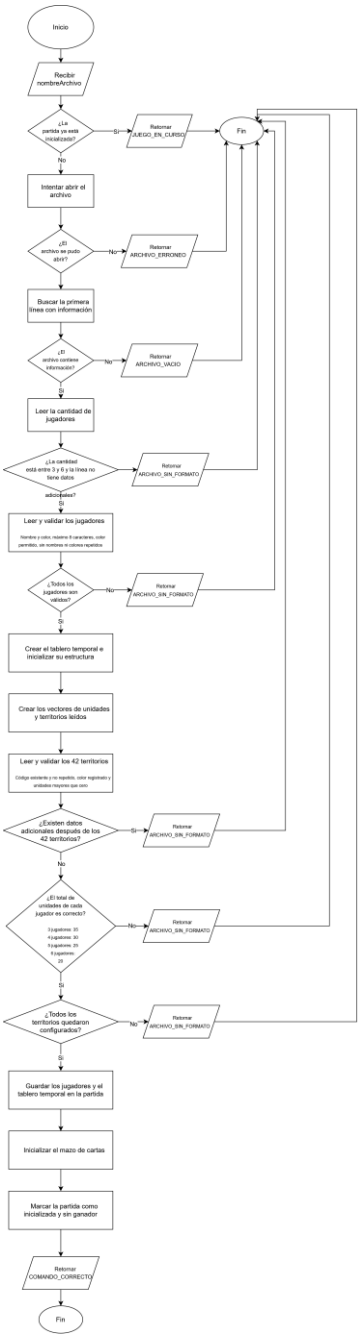


Figura 3.Diagrama de flujo de la función inicializar ubicada en Entrega0_Risk.cpp. Elaboración propia.

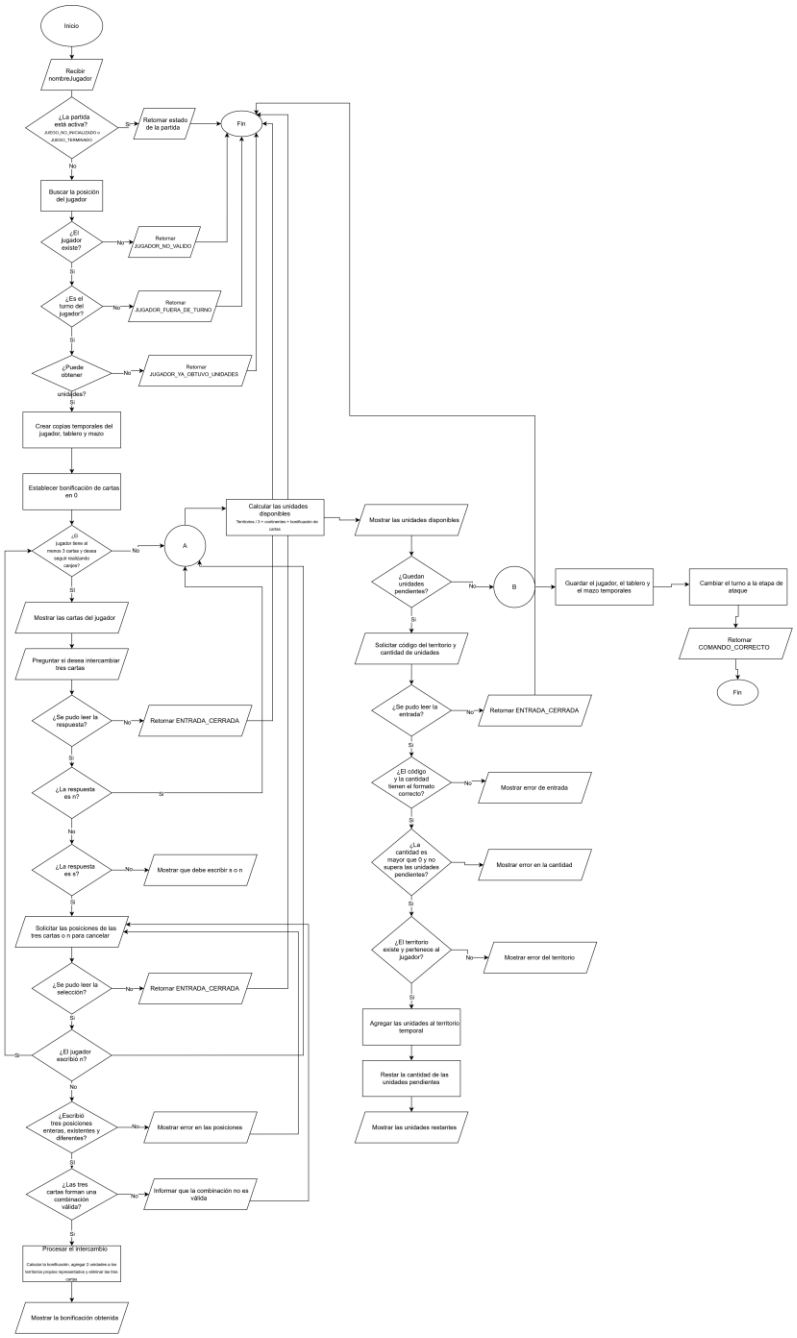


Figura 4.Diagrama de flujo de la función obtener_unidades ubicada en Entrega0_Risk.cpp. Elaboración propia.

Descripción de los Tipos Abstractos de Datos

Los Tipos Abstractos de Datos se describen mediante el conjunto mínimo de datos que forma su estado y las operaciones que componen su interfaz. Para cada variable se indican sus condiciones cuando corresponde. Cada operación presenta sus argumentos, el valor que retorna, su función, las precondiciones y las poscondiciones, siguiendo la plantilla trabajada en clase [2].

TAD Territorio

Representa un territorio del tablero, junto con su identificación, propietario, cantidad de unidades y conexiones con otros territorios.

Estado

- Código del territorio (codigo), Cadena de caracteres. Identifica el territorio.
  - Condición: Puede estar vacío únicamente mientras el territorio no haya sido configurado. En un territorio del tablero debe corresponder con uno de los 42 códigos definidos.
- Nombre del territorio (nombre), Cadena de caracteres. Guarda el nombre con el que se presenta el territorio.
  - Condición: Puede estar vacío únicamente mientras el territorio no haya sido configurado.
- Código del continente (codigoContinente), Cadena de caracteres. Identifica el continente al que pertenece el territorio.
  - Condición: En un territorio configurado debe corresponder con uno de los seis continentes del tablero.
- Color del propietario (colorPropietario), Cadena de caracteres. Identifica al jugador que controla el territorio.
  - Condición: Puede estar vacío antes de cargar la partida. Después de configurar el tablero debe corresponder con el color de uno de los jugadores.
- Cantidad de unidades (unidades), Numérico. Indica cuántas unidades se encuentran en el territorio.
  - Condición: unidades ≥ 0.
- Códigos de los vecinos (codigosVecinos), Secuencia de Cadenas de caracteres. Guarda los códigos de los territorios conectados directamente.
  - Condición: Cada código debe identificar un territorio vecino definido en el tablero.

Interfaz

Territorio(): Territorio. Crea un territorio vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un territorio con sus datos de identificación y propietario vacíos, cero unidades y sin vecinos registrados.

Territorio(nuevoCodigo, nuevoNombre, nuevoContinente): Territorio. Crea un territorio con los datos de identificación recibidos.

- Precondiciones: nuevoCodigo, nuevoNombre y nuevoContinente deben representar un territorio definido para el tablero.
- Poscondiciones: Se obtiene un territorio con el código, nombre y continente recibidos. El propietario queda vacío, la cantidad de unidades queda en cero y no se registran vecinos.

obtenerCodigo(): Cadena de caracteres. Consulta el código del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de codigo sin modificar el territorio.

obtenerNombre(): Cadena de caracteres. Consulta el nombre del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de nombre sin modificar el territorio.

obtenerCodigoContinente(): Cadena de caracteres. Consulta el código del continente al que pertenece el territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de codigoContinente sin modificar el territorio.

obtenerColorPropietario(): Cadena de caracteres. Consulta el color del propietario.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de colorPropietario sin modificar el territorio.

obtenerUnidades(): Numérico. Consulta la cantidad de unidades del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna el valor de unidades sin modificar el territorio.

asignarPropietario(nuevoColor): Vacío. Asigna directamente el propietario del territorio.

- Precondiciones: nuevoColor debe corresponder con el color que se desea cargar.
- Poscondiciones: colorPropietario queda igual a nuevoColor. Los demás datos conservan su valor.

asignarUnidades(nuevaCantidad): Vacío. Asigna directamente la cantidad de unidades del territorio.

- Precondiciones: nuevaCantidad ≥ 0.
- Poscondiciones: unidades queda igual a nuevaCantidad. Los demás datos conservan su valor.

agregarUnidades(cantidad): Booleano. Intenta agregar unidades al territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si cantidad > 0, aumenta unidades en la cantidad recibida y retorna verdadero. Si cantidad ≤ 0, retorna falso y conserva el estado anterior.

retirarUnidades(cantidad, minimoRestante): Booleano. Intenta retirar unidades sin superar el mínimo que debe permanecer.

- Precondiciones: Ninguna.
- Poscondiciones: Si cantidad > 0 y unidades - cantidad ≥ minimoRestante, descuenta la cantidad y retorna verdadero. En caso contrario, retorna falso y conserva la cantidad de unidades.

agregarVecino(codigoVecino): Vacío. Registra un territorio como vecino.

- Precondiciones: codigoVecino debe identificar un territorio conectado directamente.
- Poscondiciones: El código recibido queda agregado al final de codigosVecinos.

esVecino(codigoTerritorio): Booleano. Comprueba si un territorio se encuentra registrado como vecino.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si codigoTerritorio se encuentra en codigosVecinos. En caso contrario, retorna falso. El territorio no se modifica.

obtenerVecinos(): Secuencia de Cadenas de caracteres. Consulta todos los códigos de los territorios vecinos.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la secuencia codigosVecinos sin modificarla.

cambiarPropietario(color, cantidad): Booleano. Registra la conquista del territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio tiene cero unidades y cantidad > 0, cambia colorPropietario, agrega la cantidad recibida y retorna verdadero. En caso contrario, retorna falso y conserva el estado anterior.

TAD Continente

Representa una región del tablero que reúne varios territorios y entrega una bonificación cuando un jugador la controla completamente.

Estado

- Código del continente (codigo), Cadena de caracteres. Identifica el continente.
    - Condición: Puede estar vacío únicamente mientras el continente no haya sido configurado. En el tablero debe corresponder con uno de los seis códigos definidos.
- Nombre del continente (nombre), Cadena de caracteres. Guarda el nombre visible del continente.
    - Condición: Puede estar vacío únicamente mientras el continente no haya sido configurado.
- Bonificación (bonificacion), Numérico. Indica la cantidad de unidades adicionales que entrega el continente.
    - Condición: En un continente configurado debe corresponder con la bonificación definida en el enunciado. En un continente vacío su valor es cero.
- Territorios (territorios), Secuencia de Territorio. Contiene los territorios que forman el continente.
    - Condición: Cada territorio debe tener un código de continente igual al código del continente que lo contiene.

Interfaz

Continente(): Continente. Crea un continente vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un continente con código y nombre vacíos, bonificación igual a cero y sin territorios.

Continente(nuevoCodigo, nuevoNombre, nuevaBonificacion): Continente. Crea un continente identificado.

- Precondiciones: Los datos deben corresponder con uno de los continentes definidos en el tablero.
- Poscondiciones: Se obtiene un continente con el código, nombre y bonificación recibidos. La secuencia de territorios queda vacía.

obtenerCodigo(): Cadena de caracteres. Consulta el código del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna codigo sin modificar el continente.

obtenerNombre(): Cadena de caracteres. Consulta el nombre del continente.

- Precondiciones: Ninguna.

- Poscondiciones: Retorna nombre sin modificar el continente.

obtenerBonificacion(): Numérico. Consulta la bonificación del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna bonificacion sin modificar el continente.

agregarTerritorio(territorio): Vacío. Agrega un territorio al continente.

- Precondiciones: El código de continente del territorio debe corresponder con el continente que lo recibe.
- Poscondiciones: El territorio queda agregado al final de territorios.

obtenerTerritorios(): Secuencia de Territorio. Consulta los territorios del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la secuencia territorios sin modificarla.

estaControladoPor(colorJugador): Booleano. Comprueba si todos los territorios pertenecen al jugador indicado.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si la secuencia no está vacía y todos sus territorios tienen como propietario a colorJugador. En caso contrario, retorna falso.

buscarTerritorio(codigo): Territorio o inexistente. Busca un territorio dentro del continente.

- Precondiciones: Ninguna.
- Poscondiciones: Si encuentra un territorio con el código recibido, retorna ese territorio. Si no lo encuentra, informa que no existe.

TAD Tablero

Representa el mapa completo de Risk mediante los continentes y territorios que participan en la partida.

Estado

- Continentes (continentes), Secuencia de Continente. Contiene las regiones que forman el tablero.
    - Condición: Antes de inicializar el tablero puede estar vacía. Después de ejecutar inicializarEstructura, debe contener seis continentes y un total de 42 territorios.

Interfaz

Tablero(): Tablero. Crea un tablero vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un tablero sin continentes.

inicializarEstructura(): Vacío. Construye los continentes, territorios y conexiones del mapa.

- Precondiciones: Ninguna.

- Poscondiciones: Reemplaza el estado anterior por los seis continentes y los 42 territorios definidos para el juego.

buscarTerritorio(codigo): Territorio o inexistente. Busca un territorio en todo el tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Si encuentra un territorio con el código recibido, retorna ese territorio. Si no lo encuentra, informa que no existe.

existeTerritorio(codigo): Booleano. Comprueba si un código identifica un territorio del tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si encuentra el territorio y falso en caso contrario. El tablero conserva su estado.

configurarTerritorio(codigo, colorPropietario, unidades): Booleano. Carga el propietario y las unidades iniciales de un territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio existe, asigna el color y las unidades recibidas y retorna verdadero. Si no existe, retorna falso y conserva el tablero.

sonVecinos(codigoOrigen, codigoDestino): Booleano. Comprueba si dos territorios tienen una conexión directa.

- Precondiciones: Ninguna.
- Poscondiciones: Si ambos territorios existen, retorna el resultado de buscar codigoDestino entre los vecinos de codigoOrigen. Si alguno no existe, retorna falso. El tablero no se modifica.

contarTerritorios(colorJugador): Numérico. Cuenta los territorios pertenecientes al jugador indicado.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la cantidad de territorios cuyo propietario corresponde con colorJugador. El tablero no se modifica.

sumarUnidades(colorJugador): Numérico. Suma las unidades de todos los territorios pertenecientes a un jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la suma de unidades de los territorios cuyo propietario corresponde con colorJugador. El tablero no se modifica.

calcularBonificacionContinentes(colorJugador): Numérico. Calcula la bonificación por continentes controlados.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la suma de las bonificaciones de los continentes controlados completamente por colorJugador. El tablero no se modifica.

agregarUnidades(codigoTerritorio, colorJugador, cantidad): Booleano. Intenta agregar unidades a un territorio propio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio existe, pertenece a colorJugador y cantidad > 0, agrega las unidades y retorna verdadero. En caso contrario, retorna falso y conserva el estado anterior.

trasladarUnidades(codigoOrigen, codigoDestino, colorJugador, cantidad, minimoRestante): Booleano. Intenta trasladar unidades entre dos territorios propios vecinos.

- Precondiciones: Ninguna.
- Poscondiciones: Si ambos territorios existen, son vecinos, pertenecen a colorJugador y el origen puede conservar el mínimo indicado, descuenta la cantidad del origen, la agrega al destino y retorna verdadero. En caso contrario, retorna falso y conserva las unidades de ambos territorios.

cambiarPropietario(codigoTerritorio, nuevoColor, cantidad): Booleano. Intenta registrar la conquista de un territorio.

- Precondiciones: Ninguna.
- Poscondiciones: Si el territorio existe, pertenece a otro jugador, tiene cero unidades y la cantidad recibida es positiva, cambia su propietario, asigna las nuevas unidades y retorna verdadero. En caso contrario, retorna falso.

todosLosTerritoriosConfigurados(): Booleano. Comprueba si todos los territorios tienen propietario.

- Precondiciones: El tablero debe contener los continentes y territorios del juego.
- Poscondiciones: Retorna verdadero si ningún territorio tiene el propietario vacío. Si encuentra al menos uno sin propietario, retorna falso. El tablero no se modifica.

mostrarTablero(): Vacío. Presenta los continentes y territorios del tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Muestra el nombre, código, propietario y unidades de cada territorio. El tablero conserva su estado.

TAD Carta

Representa una carta de territorio o un comodín utilizado durante los intercambios.

Estado

- Tipo de carta (tipo), TipoCarta. Identifica la figura de la carta.
  - Condición: Debe corresponder con infantería, caballería, artillería o comodín.
- Código del territorio (codigoTerritorio), Cadena de caracteres. Identifica el territorio asociado con la carta.
  - Condición: Debe corresponder con un territorio del tablero cuando la carta no es un comodín. Para un comodín debe permanecer vacío.

Interfaz

Carta(): Carta. Crea una carta comodín.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene una carta de tipo comodín sin territorio asociado.

Carta(nuevoTipo, nuevoCodigoTerritorio): Carta. Crea una carta con los datos recibidos.

- Precondiciones: nuevoTipo debe corresponder con un tipo de carta válido. Si es un comodín, el código debe estar vacío. En los demás casos debe identificar un territorio.
- Poscondiciones: Se obtiene una carta con el tipo y el código recibidos.

obtenerTipo(): TipoCarta. Consulta el tipo de la carta.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna tipo sin modificar la carta.

obtenerCodigoTerritorio(): Cadena de caracteres. Consulta el territorio asociado con la carta.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna codigoTerritorio sin modificar la carta.

esComodin(): Booleano. Comprueba si la carta es un comodín.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si tipo corresponde con comodín y falso en caso contrario.

TAD Jugador

Representa a uno de los participantes de la partida y las cartas que ha recibido.

Estado

- Nombre del jugador (nombre), Cadena de caracteres. Identifica al jugador.
    - Condición: Al cargar una partida debe contener máximo ocho caracteres y no puede repetirse entre los jugadores.
- Color del jugador (color), Cadena de caracteres. Identifica los territorios controlados por el jugador.
    - Condición: Debe corresponder con uno de los colores permitidos y no puede repetirse dentro de la partida.
- Cartas del jugador (cartas), Secuencia de Carta. Contiene las cartas obtenidas durante la partida.
    - Condición: Cada carta debe corresponder con una carta que haya sido retirada del mazo y entregada al jugador.

Interfaz

Jugador(): Jugador. Crea un jugador vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un jugador con nombre y color vacíos y sin cartas.

Jugador(nuevoNombre, nuevoColor): Jugador. Crea un jugador identificado.

- Precondiciones: nuevoNombre y nuevoColor deben cumplir las condiciones definidas para los jugadores.
- Poscondiciones: Se obtiene un jugador con el nombre y color recibidos. La secuencia de cartas queda vacía.

obtenerNombre(): Cadena de caracteres. Consulta el nombre del jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna nombre sin modificar el jugador.

obtenerColor(): Cadena de caracteres. Consulta el color del jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna color sin modificar el jugador.

agregarCarta(nuevaCarta): Vacío. Agrega una carta al jugador.

- Precondiciones: nuevaCarta debe ser una carta válida.
- Poscondiciones: La carta recibida queda agregada al final de cartas.

obtenerCartas(): Secuencia de Carta. Permite consultar las cartas del jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la secuencia cartas. La operación de consulta no cambia el jugador.

TAD Mazo

Administra las cartas que todavía pueden entregarse y la cantidad de intercambios realizados.

Estado

- Cartas disponibles (cartasDisponibles), Secuencia de Carta. Contiene las cartas que aún no han sido entregadas.
    - Condición: Después de inicializar el mazo puede contener como máximo 44 cartas, correspondientes a 42 territorios y dos comodines.
- Cantidad de intercambios (cantidadIntercambios), Numérico. Cuenta los intercambios realizados por todos los jugadores.
    - Condición: cantidadIntercambios ≥ 0.

Interfaz

Mazo(): Mazo. Crea un mazo vacío.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un mazo sin cartas disponibles y con cero intercambios.

inicializarMazo(): Vacío. Crea las cartas utilizadas durante la partida.

- Precondiciones: Ninguna.
- Poscondiciones: cartasDisponibles queda formada por 42 cartas de territorio y dos comodines. cantidadIntercambios queda en cero.

tieneCartasDisponibles(): Booleano. Comprueba si el mazo todavía puede entregar una carta.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si cartasDisponibles contiene al menos una carta y falso si está vacía.

entregarCarta(jugador): Booleano. Intenta entregar una carta al jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Si existen cartas disponibles, selecciona una, la agrega al jugador, la elimina del mazo y retorna verdadero. Si el mazo está vacío, retorna falso y conserva su estado.

combinacionValida(cartas): Booleano. Comprueba si tres cartas forman una combinación permitida.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero cuando la secuencia contiene exactamente tres cartas del mismo tipo, una de cada tipo de ejército o dos cartas de ejército y un comodín. En los demás casos retorna falso. Las cartas no se modifican.

procesarIntercambio(jugador, tablero, posicionUno, posicionDos, posicionTres): Numérico. Valida y realiza el intercambio de tres cartas.

- Precondiciones: Ninguna.
- Poscondiciones: Si las posiciones existen, son diferentes y forman una combinación válida, aumenta cantidadIntercambios, calcula la bonificación, agrega dos unidades en los territorios propios correspondientes y elimina las tres cartas del jugador. Retorna la bonificación obtenida. Si la selección no es válida, retorna cero y conserva el estado anterior.

TAD Dados

Guarda los resultados de los dados utilizados durante una ronda de ataque.

Estado

- Resultados (resultados), Secuencia de Numéricos. Contiene los valores obtenidos en el último lanzamiento.
    - Condición: Cada resultado debe encontrarse entre 1 y 6. Después de lanzar los dados, los valores deben permanecer ordenados de mayor a menor.

Interfaz

Dados(): Dados. Crea un conjunto de dados sin resultados almacenados.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un conjunto de dados con resultados vacío.

lanzar(cantidadDados): Vacío. Genera y ordena la cantidad solicitada de resultados.

- Precondiciones: Ninguna.
- Poscondiciones: Elimina los resultados anteriores. Si cantidadDados > 0, genera esa cantidad de valores entre 1 y 6 y los ordena de mayor a menor. Si cantidadDados ≤ 0, la secuencia queda vacía.

obtenerResultados(): Secuencia de Numéricos. Consulta todos los resultados del último lanzamiento.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna resultados sin modificar los dados.

obtenerMayores(cantidad): Secuencia de Numéricos. Consulta los mayores resultados disponibles.

- Precondiciones: Ninguna.
- Poscondiciones: Si cantidad > 0, retorna los primeros resultados ordenados sin superar la cantidad disponible. Si cantidad ≤ 0, retorna una secuencia vacía. Los dados no se modifican.

limpiar(): Vacío. Elimina los resultados almacenados.

- Precondiciones: Ninguna.
- Poscondiciones: resultados queda vacío.

TAD Ataque

Controla la selección de los territorios y las rondas de dados entre el atacante y el defensor.

Estado

- Dados del atacante (dadosAtacante), Dados. Conserva los resultados del último lanzamiento del atacante.
  - Condición: Durante una ronda válida contiene entre uno y tres resultados.
- Dados del defensor (dadosDefensor), Dados. Conserva los resultados del último lanzamiento del defensor.
  - Condición: Durante una ronda válida contiene uno o dos resultados.

Interfaz

Ataque(): Ataque. Crea el controlador de una etapa de ataque.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un ataque con los dados del atacante y del defensor sin resultados almacenados.

configuracionValida(codigoOrigen, codigoDestino, colorAtacante, tablero): Booleano. Comprueba si los territorios seleccionados permiten realizar un ataque.

- Precondiciones: El tablero debe contener los territorios del juego.
- Poscondiciones: Retorna verdadero si ambos territorios existen, son vecinos, el origen pertenece al atacante, el destino pertenece a otro jugador y el origen tiene más de una unidad. En caso contrario, retorna falso. El tablero no se modifica.

ejecutar(jugadorAtacante, tablero): ResultadoAtaque. Solicita los territorios, realiza las rondas de dados y registra una conquista cuando corresponde.

- Precondiciones: El jugador debe encontrarse en la etapa de ataque y el tablero debe estar inicializado.
- Poscondiciones: Retorna un ResultadoAtaque que indica si se realizó al menos una ronda y si se conquistó un territorio. Puede modificar las unidades de los territorios y el propietario del territorio conquistado.

TAD Turno

Conserva la posición del jugador actual y la etapa que puede ejecutar.

Estado

- Posición del jugador actual (posicionJugadorActual), Numérico. Indica la posición del jugador que tiene el turno.
  - Condición: $0 \leq$ posicionJugadorActual $<$ cantidad de jugadores.
- Etapa actual (etapaActual), EtapaTurno. Indica la acción que puede ejecutar el jugador.
  - Condición: Debe corresponder con obtención de unidades, ataque o fortificación.

Interfaz

Turno(): Turno. Crea el control inicial del turno.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene un turno con la posición del jugador actual igual a cero y la etapa de obtención de unidades.

iniciar(primeraPosicion): Vacío. Establece la posición del primer jugador.

- Precondiciones: primeraPosicion debe corresponder con una posición existente dentro de la secuencia de jugadores.
- Poscondiciones: posicionJugadorActual queda igual a la posición recibida y etapaActual queda en obtención de unidades.

obtenerPosicionJugadorActual(): Numérico. Consulta la posición del jugador actual.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna posicionJugadorActual sin modificar el turno.

obtenerEtapaActual(): EtapaTurno. Consulta la etapa actual.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna etapaActual sin modificar el turno.

esTurnoDe(posicionJugador): Booleano. Comprueba si una posición corresponde con el jugador actual.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero si posicionJugador coincide con posicionJugadorActual. En caso contrario, retorna falso.

puedeObtenerUnidades(): Booleano. Comprueba si el turno se encuentra en la etapa de obtención.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero únicamente cuando etapaActual corresponde con obtención de unidades.

puedeAtacar(): Booleano. Comprueba si el turno se encuentra en la etapa de ataque.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero únicamente cuando etapaActual corresponde con ataque.

puedeFortificar(): Booleano. Comprueba si el turno se encuentra en la etapa de fortificación.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna verdadero únicamente cuando etapaActual corresponde con fortificación.

calcularNuevasUnidades(jugador, tablero, bonificacionCartas): Numérico. Calcula las unidades que obtiene un jugador.

- Precondiciones: bonificacionCartas ≥ 0.

- Poscondiciones: Retorna la división entera de los territorios controlados entre tres, más la bonificación por continentes y la bonificación por cartas. El jugador, el tablero y el turno conservan su estado.

ejecutarObtencionUnidades(jugador, tablero, codigoTerritorio, cantidad): Booleano. Intenta agregar una parte de las unidades obtenidas.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es obtención, cantidad > 0 y el territorio pertenece al jugador, agrega las unidades y retorna verdadero. En caso contrario, retorna falso y conserva el tablero.

registrarUnidadesObtenidas(): Vacío. Registra el final de la etapa de obtención.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es obtención, cambia a ataque. En cualquier otra etapa conserva su valor.

registrarAtaqueTerminado(): Vacío. Registra el final de la etapa de ataque.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es ataque, cambia a fortificación. En cualquier otra etapa conserva su valor.

ejecutarFortificacion(jugador, tablero, codigoOrigen, codigoDestino, cantidad): Booleano. Intenta trasladar unidades durante la fortificación.

- Precondiciones: Ninguna.
- Poscondiciones: Si la etapa actual es fortificación, la cantidad es positiva y el traslado es válido, modifica el tablero y retorna verdadero. En caso contrario, retorna falso y conserva el tablero.

finalizarTurno(cantidadJugadores): Vacío. Avanza al siguiente jugador.

- Precondiciones: cantidadJugadores > 0.
- Poscondiciones: Aumenta circularmente posicionJugadorActual. Después de la última posición vuelve a cero y establece la etapa de obtención de unidades.

TAD Partida

Integra los elementos del juego y controla la ejecución de los comandos del Componente 1.

Estado

- Jugadores (jugadores), Secuencia de Jugador. Contiene los participantes de la partida.
  - Condición: Antes de inicializar puede estar vacía. Después de cargar una partida válida debe contener entre tres y seis jugadores, sin nombres ni colores repetidos.
- Tablero (tablero), Tablero. Conserva los continentes, territorios, propietarios y unidades.
  - Condición: Después de inicializar la partida debe contener los seis continentes y los 42 territorios configurados.
- Turno (turno), Turno. Controla el jugador actual y la etapa que puede ejecutar.
  - Condición: La posición actual debe identificar a uno de los jugadores que todavía conserva territorios.
- Ataque (ataque), Ataque. Controla las rondas de combate.
  - Condición: Sus dados deben cumplir las condiciones establecidas por el TAD Dados.

- Mazo (mazo), Mazo. Conserva las cartas disponibles y la cantidad de intercambios.
    - Condición: Después de inicializar la partida debe contener las cartas que todavía no se han entregado.
- Partida inicializada (inicializada), Booleano. Indica si el archivo inicial se cargó correctamente.
    - Condición: Solo puede ser verdadero cuando los jugadores y los 42 territorios fueron validados y almacenados.
- Partida terminada (terminada), Booleano. Indica si ya existe un ganador.
    - Condición: Solo puede ser verdadero cuando un jugador controla los 42 territorios.
- Nombre del ganador (ganador), Cadena de caracteres. Guarda el nombre del jugador que ganó.
    - Condición: Permanece vacío mientras terminada sea falso. Cuando terminada sea verdadero debe corresponder con uno de los jugadores.

Interfaz

Partida(): Partida. Crea una partida sin información cargada.

- Precondiciones: Ninguna.
- Poscondiciones: Se obtiene una partida con inicializada y terminada en falso y con ganador vacío.

inicializar(nombreArchivo): EstadoComando. Intenta cargar la información inicial desde un archivo.

- Precondiciones: Ninguna.
- Poscondiciones: Si no existe otra partida activa y el archivo cumple el formato, guarda los jugadores, configura el tablero, inicia el turno, prepara el mazo, establece inicializada en verdadero y retorna COMANDO_CORRECTO. Si ocurre un error, retorna el estado correspondiente y no conserva una partida incompleta.

obtenerUnidades(nombreJugador): EstadoComando. Gestiona los intercambios de cartas y la distribución de unidades.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está activa, el jugador existe, tiene el turno y se encuentra en la etapa de obtención, permite realizar canjes y ubicar todas las unidades en territorios propios. Al completar el proceso guarda los cambios, avanza a ataque y retorna COMANDO_CORRECTO. Si las condiciones no se cumplen, retorna el estado correspondiente.

atacar(nombreJugador): EstadoComando. Controla la etapa de ataque del jugador.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está activa, el jugador tiene el turno y se encuentra en la etapa de ataque, permite realizar ataques. Puede modificar territorios, entregar máximo una carta y registrar al ganador. Si el jugador termina la etapa sin ganar, el turno avanza a fortificación.

fortificar(nombreJugador): EstadoComando. Realiza el traslado opcional y termina el turno.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está activa y el jugador se encuentra en la etapa de fortificación, permite realizar un traslado válido o terminar sin trasladar. Después avanza al siguiente jugador que conserve territorios y establece la etapa de obtención.

validarEstadoJuego(): EstadoComando. Comprueba si la partida puede utilizarse.

- Precondiciones: Ninguna.

- Poscondiciones: Retorna JUEGO_NO_INICIALIZADO si la partida no ha sido cargada, JUEGO_TERMINADO si ya existe un ganador o COMANDO_CORRECTO si la partida continúa. El estado no se modifica.

buscarPosicionJugador(nombreJugador): Numérico. Busca la posición de un jugador mediante su nombre.

- Precondiciones: Ninguna.
- Poscondiciones: Retorna la posición del jugador si lo encuentra. Si no existe, retorna -1. La secuencia de jugadores no se modifica.

comprobarGanador(): Booleano. Comprueba si algún jugador controla los 42 territorios.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está inicializada y encuentra un jugador con 42 territorios, establece terminada en verdadero, guarda su nombre en ganador y retorna verdadero. En caso contrario, retorna falso.

mostrarEstadoJuego(): Vacío. Presenta los jugadores, el turno actual y el tablero.

- Precondiciones: Ninguna.
- Poscondiciones: Si la partida está inicializada y no ha terminado, muestra la información actual. La partida conserva su estado.

Diagrama de relación de TAD

El diagrama debe mostrar la composición y el uso entre los TAD. Las relaciones principales se describen a continuación.

- Partida contiene una secuencia de Jugador y contiene Tablero, Turno, Ataque y Mazo.
- Tablero contiene una secuencia de Continente y cada Continente contiene una secuencia de Territorio.
- Jugador contiene una secuencia de Carta y Mazo contiene las cartas que todavía pueden entregarse.
- Ataque contiene dos objetos Dados y utiliza Jugador y Tablero durante la ejecución.
- Turno utiliza Jugador y Tablero para calcular unidades y realizar la fortificación.
- Territorio guarda el color del propietario; la relación con Jugador se establece mediante ese color.

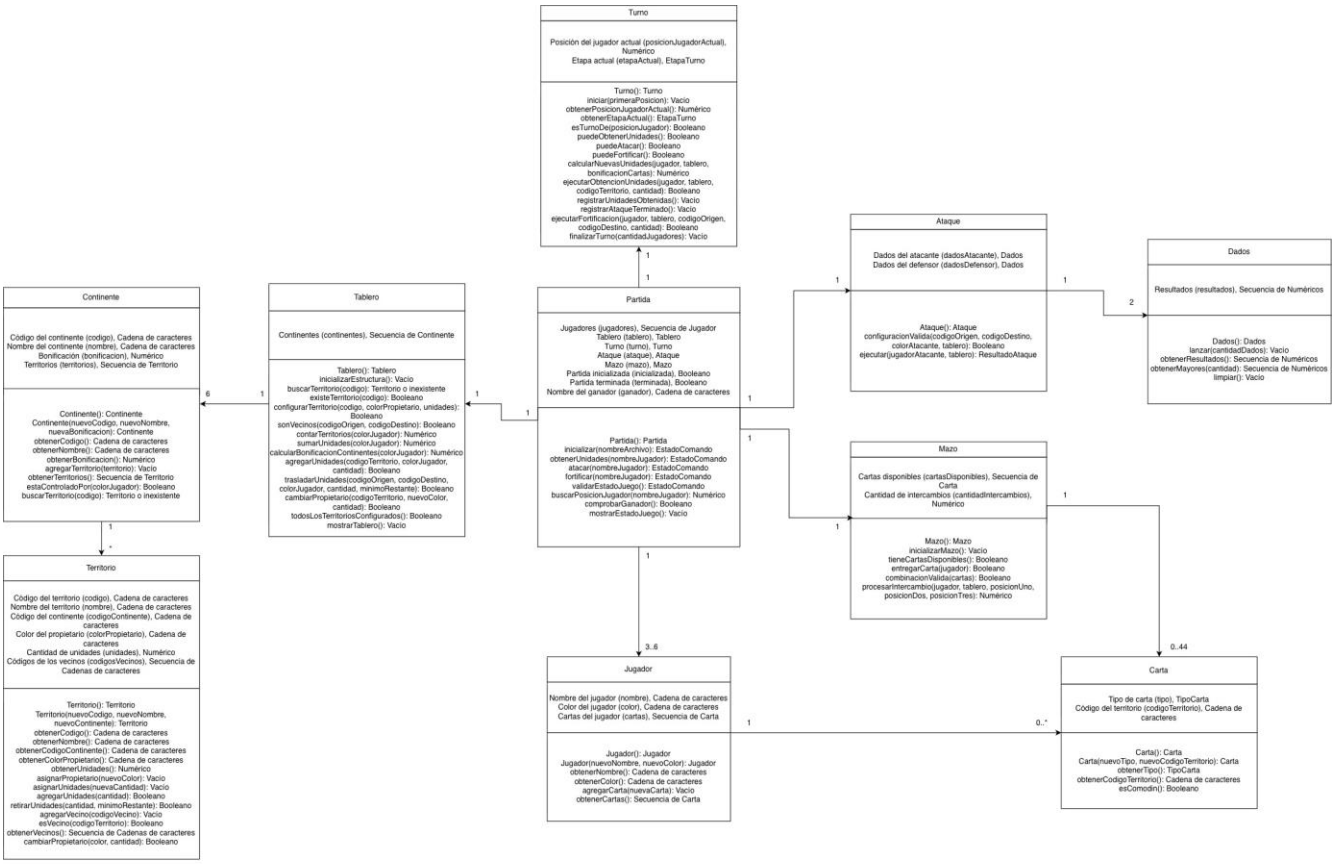


Figura 8. Relación entre los Tipos Abstractos de Datos del Componente 1. Elaboración propia.

Decisiones y aclaraciones aplicadas

- El primer jugador escrito en el archivo ocupa la posición cero y comienza la partida.
- La fortificación deja como mínimo una unidad en el territorio de origen.
- Un territorio necesita más de una unidad para iniciar o continuar un ataque.
- La cantidad de dados disminuye cuando los territorios tienen pocas unidades.
- Las cartas y sus intercambios se incluyeron en el componente obligatorio. Las misiones secretas se conservan como bono opcional y no están implementadas.
- Está pendiente ajustar el traslado posterior a una conquista: la aclaración del profesor permite escoger cero unidades, pero el código actual exige por lo menos una.

2.3 Plan de pruebas del comando atacar

El plan de pruebas se realizó para comprobar el funcionamiento del comando atacar en casos correctos y en situaciones donde el programa debe impedir la operación. Se revisaron las validaciones de la partida, el jugador, la etapa del turno y los territorios seleccionados. También se comprobó el lanzamiento de los dados, la regla de empate, la actualización de las unidades y el proceso de conquista.

Para las pruebas generales se utilizó el archivo partida.txt. Este archivo contiene tres jugadores y los 42 territorios configurados. Isa es la primera jugadora y se encuentra identificada con el color azul.

Para comprobar el caso límite de un territorio con una sola unidad se creó el archivo prueba_origen_una_unidad.txt. Este archivo conserva el total inicial de unidades del jugador, pero cambia la distribución de dos territorios: 1.2 queda con una unidad y 1.5 queda con cinco unidades.

Los resultados de los dados no se establecieron de antemano porque el programa los genera de manera aleatoria. En estas pruebas se revisó que cada valor estuviera entre 1 y 6, que los resultados aparecieran ordenados de mayor a menor y que las pérdidas correspondieran con cada comparación. En caso de empate debía perder una unidad el atacante.

2.3.1. Planificación de las pruebas

ID	Tipo de caso	Elemento analizado	Preparación o entrada	Resultado esperado
P01	Error	Partida no inicializada	Ejecutar atacar Isa antes de inicializar una partida.	El programa informa que la partida no ha sido inicializada. No solicita territorios ni lanza dados.
P02	Error	Jugador inexistente	Inicializar partida.txt y ejecutar atacar Pedro.	El programa informa que Pedro no forma parte de la partida y no modifica el tablero.
P03	Error	Jugador fuera de turno	Inicializar partida.txt y ejecutar atacar Saul, aunque el turno corresponde a Isa.	El programa informa que no es el turno de Saul y no permite comenzar el ataque.
P04	Error	Etapa incorrecta	Inicializar partida.txt y ejecutar atacar Isa antes de utilizar obtener_unidades.	El programa informa que Isa todavía no ha ubicado sus unidades y no comienza el ataque.
P05	Error	Territorio de origen ajeno	Avanzar hasta la etapa de ataque de Isa y seleccionar 1.1 como origen y 1.2 como destino.	La configuración se rechaza porque 1.1 pertenece al jugador verde. No se lanzan dados.
P06	Error	Territorio de destino propio	Avanzar hasta la etapa de ataque de Isa y seleccionar 1.2 como origen y 1.5 como destino.	La configuración se rechaza porque los dos territorios pertenecen a Isa. No se lanzan dados.
P07	Error	Territorios no vecinos	Avanzar hasta la etapa de ataque de Isa y seleccionar 1.2 como origen y 4.1 como destino.	La configuración se rechaza porque los territorios no tienen una conexión directa.
P08	Límite	Origen con una unidad	Inicializar prueba_origen_una_unidad.txt, ubicar las unidades obtenidas en 1.5 e intentar atacar desde 1.2 hacia 1.1.	La configuración se rechaza porque el territorio de origen solamente conserva una unidad.
P09	Normal	Ataque válido sin conquista	Inicializar partida.txt, ubicar cuatro unidades en 1.2 y atacar desde 1.2 hacia 1.1. Finalizar después de la primera ronda.	Se lanzan los dados permitidos, se actualizan las unidades y el territorio defensor conserva su propietario si todavía tiene unidades.
P10	Límite	Cantidad de dados y empate	Realizar un ataque desde 1.2, que tiene siete unidades, hacia 1.1, que tiene tres. Continuar hasta observar dos dados con el mismo valor.	El atacante lanza tres dados y el defensor dos. Cuando los valores comparados son iguales, pierde una unidad el atacante.
P11	Normal y límite	Conquista, traslado de cero	Continuar el ataque desde 1.2 hacia 1.1 hasta que el defensor llegue a cero unidades. Seleccionar cero como cantidad para trasladar.	El territorio cambia de propietario, se acepta el traslado de cero unidades, el origen conserva sus unidades y el jugador recibe una carta.

		unidades y carta		
P12	Error	Territorio inexistente	Avanzar hasta la etapa de ataque de Isa y seleccionar 1.2 como origen y 9.9 como destino.	La configuración se rechaza porque 9.9 no identifica un territorio del tablero. No se lanzan dados.

Ejecución de pruebas

Las pruebas se ejecutaron compilando todos los archivos de implementación con el siguiente comando:

```
g++ -std=c++17 *.cpp
```

Después de compilar, el programa se ejecutó mediante:

```
./a.out
```

Los casos P01 a P07, P09 y P12 se realizaron utilizando partida.txt. Para P08 se utilizó prueba_origen_una_unidad.txt. Las pruebas P10 y P11 se ejecutaron en una misma etapa de ataque, debido a que durante las rondas se presentó un empate y posteriormente se conquistó el territorio.

Ejecución de pruebas del comando atacar

ID	Elemento analizado	Resultado esperado	Resultado obtenido	¿Coincide?	Explicación
P01	Partida no inicializada	El programa debe informar que la partida no ha sido inicializada y no debe comenzar el ataque.	Se mostró: (Juego no inicializado) Esta partida no ha sido inicializada correctamente. No se solicitaron territorios ni se lanzaron dados.	Sí	El comando comprobó el estado de la partida antes de intentar ejecutar el ataque.
P02	Jugador inexistente	El programa debe informar que el jugador no forma parte de la partida y conservar el tablero.	Se mostró: (Jugador no válido) El jugador Pedro no forma parte de esta partida.	Sí	Después de validar la partida, el programa buscó el nombre recibido y detuvo el comando porque no encontró al jugador.
P03	Jugador fuera de turno	El programa debe impedir que un jugador diferente al actual realice el ataque.	Se mostró: (Jugador fuera de turno) No es el turno del jugador Saul.	Sí	Saul pertenece a la partida, pero el turno actual correspondía a Isa. Por esta razón no se inició el ataque.
P04	Etapa incorrecta	El programa debe impedir el ataque cuando el jugador todavía no ha obtenido y ubicado sus unidades.	Se mostró: (Jugador no ha ubicado unidades) El jugador Isa no ha ejecutado el comando obtener_unidades.	Sí	Isa tenía el turno, pero todavía se encontraba en la etapa de obtención de unidades.
P05	Territorio de origen ajeno	El programa debe rechazar la configuración cuando el territorio de origen no pertenece al atacante.	Se seleccionó 1.1 como origen y 1.2 como destino. El programa mostró: La configuracion del ataque no es valida.	Sí	El territorio 1.1 pertenecía al jugador verde, mientras que Isa controlaba el color azul. Por eso no podía utilizarlo como origen.
P06	Territorio de destino propio	El programa debe rechazar un ataque dirigido contra otro territorio del mismo jugador.	Se seleccionó 1.2 como origen y 1.5 como destino. El programa mostró: La configuracion del ataque no es valida.	Sí	Los dos territorios pertenecían a Isa. El destino de un ataque debe pertenecer a otro jugador.
P07	Territorios no vecinos	El programa debe rechazar el ataque cuando los territorios no tienen una conexión directa.	Se seleccionó 1.2 como origen y 4.1 como destino. El programa mostró: La configuracion del ataque no es valida.	Sí	Aunque el origen pertenecía a Isa y el destino era enemigo, los dos territorios no eran vecinos.
P08	Origen con una unidad	El ataque no debe comenzar cuando el territorio de origen solamente conserva una unidad.	Se cargó prueba_origen_una_unidad.txt, donde 1.2 pertenecía a Isa y tenía una unidad. Al intentar atacar desde 1.2 hacia 1.1, se mostró: La configuracion del ataque no es valida. No se lanzaron dados.	Sí	La validación comprobó que el territorio de origen no tenía más de una unidad y detuvo el ataque antes de modificar el tablero.
P09	Ataque válido sin conquista	Deben lanzarse los dados permitidos, ordenarse de mayor a menor y actualizarse las unidades. Si el defensor conserva	El atacante obtuvo 5 5 1 y el defensor 4 2. El defensor perdió dos unidades. El origen conservó 7 unidades y el destino quedó con 1. El jugador decidió no continuar el ataque.	Sí	Los resultados se ordenaron y compararon por posición. Los dos dados mayores del atacante superaron a los del defensor, pero el territorio no fue conquistado porque todavía conservaba una unidad.

		unidades, el territorio no debe cambiar de propietario.			
P10	Empate y cantidad de dados	La cantidad de dados debe depender de las unidades disponibles. Si los valores comparados son iguales, debe perder una unidad el atacante.	El atacante lanzó 6 5 1 y el defensor 6 1. En la primera comparación ocurrió un empate de 6 contra 6 y el programa indicó: El atacante pierde una unidad.	Sí	El atacante lanzó tres dados y el defensor dos según sus unidades disponibles. En el empate se aplicó correctamente la regla que favorece al defensor.
P11	Conquista, traslado y carta	Cuando el defensor llegue a cero unidades, el territorio debe cambiar de propietario, permitir el traslado desde cero unidades y entregar máximo una carta durante la etapa.	En la segunda ronda el atacante obtuvo 5 3 1 y el defensor 2 1. El defensor perdió sus dos unidades restantes. El programa permitió trasladar entre 0 y 5 unidades, el jugador eligió 0, Alaska cambió a propietario azul y se entregó una carta.	Sí	El territorio cambió de propietario después de quedar sin unidades. La corrección permitió trasladar cero unidades y estado_juego confirmó que Alaska quedó con propietario azul y cero unidades. También se entregó una carta por la conquista.
P12	Territorio inexistente	El programa debe rechazar la configuración cuando alguno de los códigos no identifica un territorio del tablero.	Se seleccionó 1.2 como origen y 9.9 como destino. El programa mostró: La configuracion del ataque no es valida. No se lanzaron dados.	Sí	La búsqueda no encontró un territorio con el código 9.9, por lo que el ataque se detuvo antes de modificar las unidades.

3 Referencias

[1] Pontificia Universidad Javeriana, Departamento de Ingeniería de Sistemas. Proyecto del curso 2026-30. Estructuras de Datos, 2026.

[2] A. Castro Martínez. Tipo Abstracto de Dato TAD. Material de clase de Estructuras de Datos, Pontificia Universidad Javeriana, 2023.

[3] A. Castro Martínez. Secuencias; Vectores y Listas. Material de clase de Estructuras de Datos, Pontificia Universidad Javeriana, 2023.

[4] S. L. Cruz Romero, I. Hermosa Losada y A. Macías Barrios. Taller 2 Estructuras Lineales Sistema de Batallas Pokémon. Pontificia Universidad Javeriana, 2026.

4 Registro del apoyo de herramientas de inteligencia artificial

Durante el desarrollo se utilizaron herramientas de inteligencia artificial como apoyo para revisar la lógica, localizar diferencias entre el código y el enunciado, explicar elementos de C++ y organizar la documentación. Las sugerencias se compararon con el código del grupo, el material de clase y las aclaraciones del profesor antes de aceptarlas.

4.1 Explicación de elementos de C++

Se pidió explicar referencias, const, punteros, vectores, rand, srand, time y lectura con getline e istringstream. Estas explicaciones se utilizaron para comprender el código y preparar la sustentación.

4.2 Apoyo en la documentación

Se utilizó la herramienta para comparar el avance con el formato oficial y el Taller 2, mejorar la redacción y preparar tablas, marcadores de figuras y casos de prueba. Los resultados obtenidos permanecen pendientes hasta que sean ejecutados y evidenciados por el grupo.